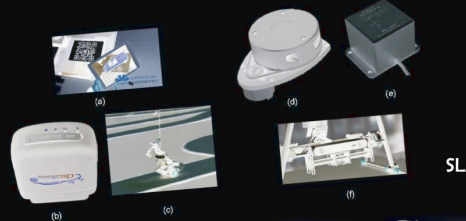
Vision-based Simultaneous Localization And Mapping

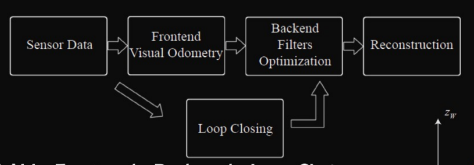
Dr. Xiang Gao

Ph.D of Tsinghua University, Postdoc at Technical University of Munich
idriverplus.com

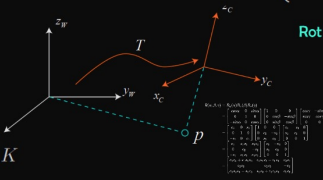
课程回放：https://www.shenlanxueyuan.com/course/386

Basic Knowledge on Visual SLAM
From Theory to Practice





SLAM = Front end + Back end + Loop Closing



Sensors:
GNSS QR Code Trajectory
Camera Lidar IMU Odom

Applications:
UAV Self-driving Robots AR/VR



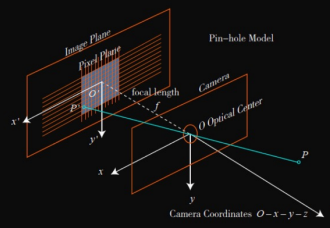
Cameras: Mono Stereo RGBD
No depth Passive Active

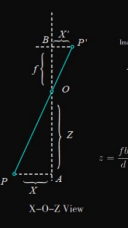
State Estimation Problem

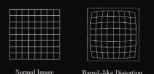
$$\begin{cases} \mathbf{x}_k = f(\mathbf{x}_{k-1}, \mathbf{u}_k, \mathbf{w}_k), & k = 1, \cdots, K \\ \mathbf{z}_{k,j} = h(\mathbf{y}_j, \mathbf{x}_k, \mathbf{v}_{k,j}), & (k,j) \in \mathcal{O} \end{cases}$$

$$\begin{bmatrix} x_1 \\ x_2 \\ \theta \end{bmatrix}_k = \begin{bmatrix} x_1 \\ x_2 \\ \theta \end{bmatrix}_{k-1} + \begin{bmatrix} \Delta x_1 \\ \Delta x_2 \\ \Delta\theta \end{bmatrix}_k + \mathbf{w}_k, \quad \begin{bmatrix} r_{k,j} \\ \phi_{k,j} \end{bmatrix} = \begin{bmatrix} \sqrt{(y_{1,j}-x_{1,k})^2 + (y_{2,j}-x_{2,k})^2} \\ \arctan\left(\frac{y_{2,j}-x_{2,k}}{y_{1,j}-x_{1,k}}\right) \end{bmatrix}$$

Pinhole Camera Model



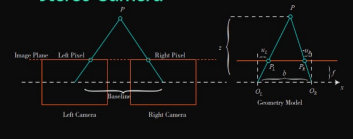




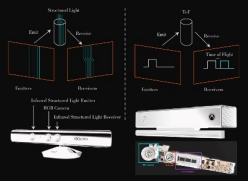
$$\begin{pmatrix} u \\ v \\ 1 \end{pmatrix} = \frac{1}{Z}\begin{pmatrix} f_x & 0 & c_x \\ 0 & f_y & c_y \\ 0 & 0 & 1 \end{pmatrix}\begin{pmatrix} X \\ Y \\ Z \end{pmatrix} \triangleq \frac{1}{Z}\mathbf{K}\mathbf{P}.$$

$$\begin{cases} x_{\text{distorted}} = x(1 + k_1 r^2 + k_2 r^4 + k_3 r^6) + 2p_1 xy + p_2(r^2 + 2x^2) \\ y_{\text{distorted}} = y(1 + k_1 r^2 + k_2 r^4 + k_3 r^6) + p_1(r^2 + 2y^2) + 2p_2 xy \end{cases}$$

Stereo Camera



$$z = \frac{fb}{d}, \quad d \triangleq u_L - u_R.$$

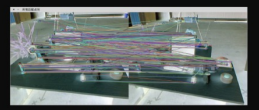


RGBD Camera

Feature Matching



ORB features



Brute-force Matching



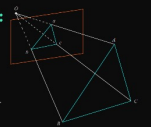
3D-2D: PnP

DLT:

$$s\begin{pmatrix} u_1 \\ v_1 \\ 1 \end{pmatrix} = \underbrace{\begin{pmatrix} t_1 & t_2 & t_3 & t_4 \\ t_5 & t_6 & t_7 & t_8 \\ t_9 & t_{10} & t_{11} & t_{12} \end{pmatrix}}_{[\mathbf{R}|\mathbf{t}]}\begin{pmatrix} X \\ Y \\ Z \\ 1 \end{pmatrix}.$$

$$\mathbf{R} \leftarrow (\mathbf{R}\mathbf{R}^T)^{-\frac{1}{2}}\mathbf{R}.$$

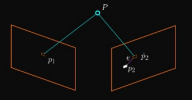
P3P:



$$(1-u)y^2 - ux^2 - 2\cos\langle b,c\rangle y + 2ux\cos\langle a,b\rangle + 1 = 0$$
$$(1-w)x^2 - wy^2 - 2\cos\langle a,c\rangle x + 2wx\cos\langle a,b\rangle + 1 = 0.$$

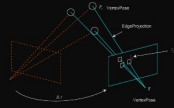
Pose only BA:

$$\mathbf{T}^* = \arg\min_{\mathbf{T}} \frac{1}{2}\sum_{i=1}^{n}\left\|\mathbf{u}_i - \frac{1}{s_i}\mathbf{K}\mathbf{T}\mathbf{P}_i\right\|_2^2.$$

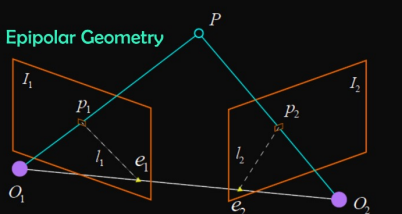


$$\frac{\partial \mathbf{e}}{\partial \boldsymbol{\xi}} = \lim_{\delta\boldsymbol{\xi}\to 0}\frac{\mathbf{e}(\delta\boldsymbol{\xi}\oplus\boldsymbol{\xi}) - \mathbf{e}(\boldsymbol{\xi})}{\delta\boldsymbol{\xi}} = \frac{\partial \mathbf{e}}{\partial \mathbf{P}'}\frac{\partial \mathbf{P}'}{\partial \boldsymbol{\xi}}$$

$$\frac{\partial \mathbf{e}}{\partial \boldsymbol{\xi}} = -\begin{bmatrix} \frac{f_x}{Z'} & 0 & -\frac{f_x X'}{Z'^2} & -\frac{f_x X' Y'}{Z'^2} & f_x + \frac{f_x X'^2}{Z'^2} & -\frac{f_x Y'}{Z'} \\ 0 & \frac{f_y}{Z'} & -\frac{f_y Y'}{Z'^2} & -f_y - \frac{f_y Y'^2}{Z'^2} & \frac{f_y X' Y'}{Z'^2} & \frac{f_y X'}{Z'} \end{bmatrix}$$



2D-2D: Epipolar Geometry



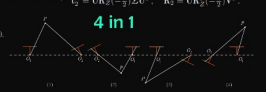
$$\mathbf{x}_2^T \mathbf{t}^\wedge \mathbf{R}\mathbf{x}_1 = 0.$$

1. Find $\mathbf{E}$ or $\mathbf{F}$ based on the pixel positions of the matched points.
2. Find $\mathbf{R}, \mathbf{t}$ based on $\mathbf{E}$ or $\mathbf{F}$.

$$\mathbf{E} = \mathbf{U}\Sigma\mathbf{V}^T, \quad \mathbf{t}_1^\wedge = \mathbf{U}\mathbf{R}_Z(\tfrac{\pi}{2})\Sigma\mathbf{U}^T, \quad \mathbf{R}_1 = \mathbf{U}\mathbf{R}_Z^T(\tfrac{\pi}{2})\mathbf{V}^T$$
$$\mathbf{t}_2^\wedge = \mathbf{U}\mathbf{R}_Z(-\tfrac{\pi}{2})\Sigma\mathbf{U}^T, \quad \mathbf{R}_2 = \mathbf{U}\mathbf{R}_Z^T(-\tfrac{\pi}{2})\mathbf{V}^T.$$

$$\begin{pmatrix} u_1^1 u_2^1 & u_1^1 v_2^1 & u_1^1 & v_1^1 u_2^1 & v_1^1 v_2^1 & v_1^1 & u_2^1 & v_2^1 & 1 \\ u_1^2 u_2^2 & u_1^2 v_2^2 & u_1^2 & v_1^2 u_2^2 & v_1^2 v_2^2 & v_1^2 & u_2^2 & v_2^2 & 1 \\ \vdots & \vdots & \vdots & \vdots & \vdots & \vdots & \vdots & \vdots & \vdots \\ u_1^8 u_2^8 & u_1^8 v_2^8 & u_1^8 & v_1^8 u_2^8 & v_1^8 v_2^8 & v_1^8 & u_2^8 & v_2^8 & 1 \end{pmatrix}\begin{pmatrix} e_1 \\ e_2 \\ e_3 \\ e_4 \\ e_5 \\ e_6 \\ e_7 \\ e_8 \\ e_9 \end{pmatrix} = 0.$$

4 in 1



Triangulation



$$s_2\mathbf{x}_2^\wedge\mathbf{x}_2 = 0 = s_1\mathbf{x}_2^\wedge\mathbf{R}\mathbf{x}_1 + \mathbf{x}_2^\wedge\mathbf{t}.$$

Homography

$$\mathbf{p}_2 \simeq \mathbf{K}(\mathbf{R}\mathbf{P} + \mathbf{t})$$
$$\simeq \mathbf{K}\left(\mathbf{R} - \frac{\mathbf{t}\mathbf{n}^T}{d}\right)\mathbf{P}$$
$$\simeq \underbrace{\mathbf{K}\left(\mathbf{R} - \frac{\mathbf{t}\mathbf{n}^T}{d}\right)\mathbf{K}^{-1}}_{\mathbf{H}}\mathbf{p}_1.$$

$$\begin{pmatrix} u_1^1 & v_1^1 & 1 & 0 & 0 & 0 & -u_1^1 u_2^1 & -v_1^1 u_2^1 \\ 0 & 0 & 0 & u_1^1 & v_1^1 & 1 & -u_1^1 v_2^1 & -v_1^1 v_2^1 \\ u_1^2 & v_1^2 & 1 & 0 & 0 & 0 & -u_1^2 u_2^2 & -v_1^2 u_2^2 \\ 0 & 0 & 0 & u_1^2 & v_1^2 & 1 & -u_1^2 v_2^2 & -v_1^2 v_2^2 \\ \vdots & \vdots & \vdots & \vdots & \vdots & \vdots & \vdots & \vdots \\ u_1^4 & v_1^4 & 1 & 0 & 0 & 0 & -u_1^4 u_2^4 & -v_1^4 u_2^4 \\ 0 & 0 & 0 & u_1^4 & v_1^4 & 1 & -u_1^4 v_2^4 & -v_1^4 v_2^4 \end{pmatrix}\begin{pmatrix} h_1 \\ h_2 \\ h_3 \\ h_4 \\ h_5 \\ h_6 \\ h_7 \\ h_8 \end{pmatrix} = \begin{pmatrix} u_2^1 \\ v_2^1 \\ u_2^2 \\ v_2^2 \\ \vdots \\ u_2^4 \\ v_2^4 \end{pmatrix}$$

3D-3D: ICP

$$\min_{\mathbf{R},\mathbf{t}} \frac{1}{2}\sum_{i=1}^{n}\|(\mathbf{p}_i - (\mathbf{R}\mathbf{p}_i' + \mathbf{t})\|_2^2.$$

SVD

$$\mathbf{W} = \sum_{i=1}^{n}\mathbf{q}_i\mathbf{q}_i'^T.$$

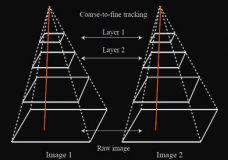
$$\mathbf{W} = \mathbf{U}\Sigma\mathbf{V}^T.$$

$$\mathbf{R} = \mathbf{U}\mathbf{V}^T, \quad \mathbf{t}^* = \mathbf{p} - \mathbf{R}\mathbf{p}'.$$

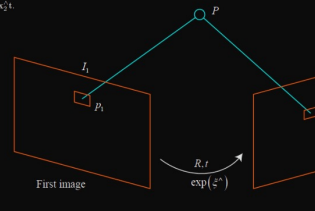
Optm:

$$\min_{\boldsymbol{\xi}} = \frac{1}{2}\sum_{i=1}^{n}\|(\mathbf{p}_i - \exp(\boldsymbol{\xi}^\wedge)\mathbf{p}_i'\|_2^2.$$

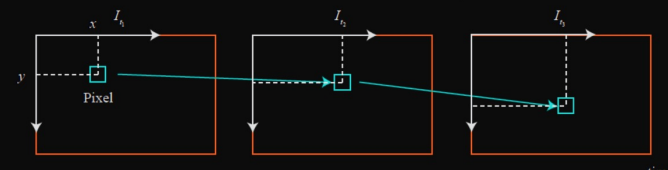
$$\frac{\partial \mathbf{e}}{\partial \boldsymbol{\xi}} = -(\exp(\boldsymbol{\xi}^\wedge)\mathbf{p}_i')^\odot.$$



Direct Method



Optical Flow

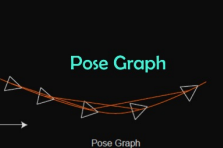


Constant Grayscale Assumption

$$\mathbf{I}(x + dx, y + dy, t + dt) = \mathbf{I}(x, y, t).$$

$$\begin{bmatrix} \mathbf{I}_x & \mathbf{I}_y \end{bmatrix}\begin{bmatrix} u \\ v \end{bmatrix} = -\mathbf{I}_t.$$



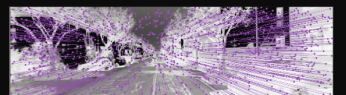


$$\mathbf{p}_1 = \begin{bmatrix} u \\ v \\ 1 \end{bmatrix}_1 = \frac{1}{Z_1}\mathbf{K}\mathbf{P},$$

$$\mathbf{p}_2 = \begin{bmatrix} u \\ v \\ 1 \end{bmatrix}_2 = \frac{1}{Z_2}\mathbf{K}(\mathbf{R}\mathbf{P} + \mathbf{t}) = \frac{1}{Z_2}\mathbf{K}(\mathbf{T}\mathbf{P})_{1:3},$$

$$\frac{\partial \mathbf{u}}{\partial \mathbf{q}_1} = \begin{bmatrix} \frac{\partial u}{\partial X} & \frac{\partial u}{\partial Y} & \frac{\partial u}{\partial Z} \\ \frac{\partial v}{\partial X} & \frac{\partial v}{\partial Y} & \frac{\partial v}{\partial Z} \end{bmatrix} = \begin{bmatrix} \frac{f_x}{Z'} & 0 & -\frac{f_x X'}{Z'^2} \\ 0 & \frac{f_y}{Z'} & -\frac{f_y Y'}{Z'^2} \end{bmatrix}.$$

$$\frac{\partial \mathbf{u}}{\partial \delta\boldsymbol{\xi}} = \begin{bmatrix} \frac{f_x}{Z'} & 0 & -\frac{f_x X'}{Z'^2} & -\frac{f_x X' Y'}{Z'^2} & f_x + \frac{f_x X'^2}{Z'^2} & -\frac{f_x Y'}{Z'} \\ 0 & \frac{f_y}{Z'} & -\frac{f_y Y'}{Z'^2} & -f_y - \frac{f_y Y'^2}{Z'^2} & \frac{f_y X' Y'}{Z'^2} & \frac{f_y X'}{Z'} \end{bmatrix}$$



Rotation

$$\mathrm{SO}(n) = \{\mathbf{R} \in \mathbb{R}^{n\times n} | \mathbf{R}\mathbf{R}^T = \mathbf{I}, \det(\mathbf{R}) = 1\}.$$

Transform

$$\mathrm{SE}(3) = \left\{\mathbf{T} = \begin{bmatrix} \mathbf{R} & \mathbf{t} \\ \mathbf{0}^T & 1 \end{bmatrix} \in \mathbb{R}^{4\times 4} | \mathbf{R} \in \mathrm{SO}(3), \mathbf{t} \in \mathbb{R}^3\right\}$$

$$\mathfrak{so}(3) = \left\{\boldsymbol{\phi} \in \mathbb{R}^3 \text{ or } \boldsymbol{\Phi} = \boldsymbol{\phi}^\wedge \in \mathbb{R}^{3\times 3}\right\}.$$

$$\mathfrak{se}(3) = \left\{\boldsymbol{\xi} = \begin{bmatrix} \boldsymbol{\rho} \\ \boldsymbol{\phi} \end{bmatrix} \in \mathbb{R}^6, \boldsymbol{\rho} \in \mathbb{R}^3, \boldsymbol{\phi} \in \mathfrak{so}(3), \boldsymbol{\xi}^\wedge = \begin{bmatrix} \boldsymbol{\phi}^\wedge & \boldsymbol{\rho} \\ \mathbf{0}^T & 0 \end{bmatrix} \in \mathbb{R}^{4\times 4}\right\}.$$

Skew-symmetric

$$\mathbf{a}^\wedge = \mathbf{A} = \begin{bmatrix} 0 & -a_3 & a_2 \\ a_3 & 0 & -a_1 \\ -a_2 & a_1 & 0 \end{bmatrix}, \quad \mathbf{A}^\vee = \mathbf{a}.$$

$\wedge$ in SE(3)

$$\boldsymbol{\xi}^\wedge = \begin{bmatrix} \boldsymbol{\phi}^\wedge & \boldsymbol{\rho} \\ \mathbf{0}^T & 0 \end{bmatrix} \in \mathbb{R}^{4\times 4}.$$

$$\mathbf{n}^\wedge\mathbf{n}^\wedge\mathbf{n}^\wedge = \mathbf{n}^\wedge(\mathbf{n}\mathbf{n}^T - \mathbf{I}) = \underbrace{\mathbf{n}\mathbf{n}^T}_{\text{zero}}\mathbf{n}^\wedge - \mathbf{n}^\wedge = -\mathbf{n}^\wedge.$$

$$\mathbf{n}^\wedge\mathbf{n}^\wedge = \begin{bmatrix} -n_2^2 - n_3^2 & n_1 n_2 & n_1 n_3 \\ n_1 n_2 & -n_1^2 - n_3^2 & n_2 n_3 \\ n_1 n_3 & n_2 n_3 & -n_1^2 - n_2^2 \end{bmatrix} = \mathbf{n}\mathbf{n}^T - \mathbf{I},$$

Exp map:

$$\exp(\boldsymbol{\phi}^\wedge) = \exp(\theta\mathbf{n}^\wedge) = \sum_{n=0}^{\infty}\frac{1}{n!}(\theta\mathbf{n}^\wedge)^n$$
$$= \mathbf{I} + \theta\mathbf{n}^\wedge + \frac{1}{2!}\theta^2\mathbf{n}^\wedge\mathbf{n}^\wedge + \frac{1}{3!}\theta^3\mathbf{n}^\wedge\mathbf{n}^\wedge\mathbf{n}^\wedge + \frac{1}{4!}\theta^4(\mathbf{n}^\wedge)^4 + \ldots$$
$$= \mathbf{n}\mathbf{n}^T - \mathbf{n}^\wedge\mathbf{n}^\wedge + \theta\mathbf{n}^\wedge + \frac{1}{2!}\theta^2\mathbf{n}^\wedge\mathbf{n}^\wedge - \frac{1}{3!}\theta^3\mathbf{n}^\wedge - \frac{1}{4!}\theta^4(\mathbf{n}^\wedge)^2 + \ldots$$
$$= \mathbf{n}\mathbf{n}^T + \underbrace{\left(\theta - \frac{1}{3!}\theta^3 + \frac{1}{5!}\theta^5 - \ldots\right)}_{\sin\theta}\mathbf{n}^\wedge - \underbrace{\left(1 - \frac{1}{2!}\theta^2 + \frac{1}{4!}\theta^4 - \ldots\right)}_{\cos\theta}\mathbf{n}^\wedge\mathbf{n}^\wedge$$
$$= \mathbf{n}^\wedge\mathbf{n}^\wedge + \mathbf{I} + \sin\theta\mathbf{n}^\wedge - \cos\theta\mathbf{n}^\wedge\mathbf{n}^\wedge$$
$$= (1 - \cos\theta)\mathbf{n}^\wedge\mathbf{n}^\wedge + \mathbf{I} + \sin\theta\mathbf{n}^\wedge$$
$$= \cos\theta\mathbf{I} + (1 - \cos\theta)\mathbf{n}\mathbf{n}^T + \sin\theta\mathbf{n}^\wedge.$$

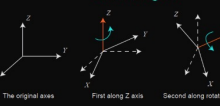
Rot Matrix $\leftrightarrow$ Rot Vector

$$\theta = \arccos\left(\frac{\mathrm{tr}(\mathbf{R}) - 1}{2}\right). \quad \mathbf{R}\mathbf{n} = \mathbf{n}.$$

$$\mathbf{R} = \mathbf{v}\mathbf{v}^T + s^2\mathbf{I} + 2s\mathbf{v}^\wedge, \quad (\mathbf{v}^\wedge)^2.$$

$$\begin{cases} \theta = 2\arccos q_0 \\ [n_x, n_y, n_z]^T = [q_1, q_2, q_3]^T/\sin\frac{\theta}{2} \end{cases}$$

Euler Angles



Quarternion

$$\mathbf{q} = q_0 + q_1 i + q_2 j + q_3 k,$$
$$\mathbf{q} = [s, \mathbf{v}]^T, \quad s = q_0 \in \mathbb{R}, \quad \mathbf{v} = [q_1, q_2, q_3]^T \in \mathbb{R}^3,$$
$$\mathbf{q}_a\mathbf{q}_b = \begin{bmatrix} s_a s_b - \mathbf{v}_a^T\mathbf{v}_b, s_a\mathbf{v}_b + s_b\mathbf{v}_a + \mathbf{v}_a \times \mathbf{v}_b \end{bmatrix}^T.$$

Rodrigues' Formula

$$\exp(\boldsymbol{\xi}^\wedge) = \begin{bmatrix} \sum_{n=0}^{\infty}\frac{1}{n!}(\boldsymbol{\phi}^\wedge)^n & \sum_{n=0}^{\infty}\frac{1}{(n+1)!}(\boldsymbol{\phi}^\wedge)^n\boldsymbol{\rho} \\ \mathbf{0}^T & 1 \end{bmatrix}$$
$$\triangleq \begin{bmatrix} \mathbf{R} & \mathbf{J}\boldsymbol{\rho} \\ \mathbf{0}^T & 1 \end{bmatrix} = \mathbf{T}. \quad \mathbf{J} = \frac{\sin\theta}{\theta}\mathbf{I} + \left(1 - \frac{\sin\theta}{\theta}\right)\mathbf{a}\mathbf{a}^T + \frac{1 - \cos\theta}{\theta}\mathbf{a}^\wedge.$$

Baker-Campbell-Hausdorff Formula:

$$\ln(\exp(\boldsymbol{\phi}_1^\wedge)\exp(\boldsymbol{\phi}_2^\wedge))^\vee \approx \begin{cases} \mathbf{J}_l(\boldsymbol{\phi}_2)^{-1}\boldsymbol{\phi}_1 + \boldsymbol{\phi}_2 & \text{when } \boldsymbol{\phi}_1 \text{ is a small amount,} \\ \mathbf{J}_r(\boldsymbol{\phi}_1)^{-1}\boldsymbol{\phi}_2 + \boldsymbol{\phi}_1 & \text{when } \boldsymbol{\phi}_2 \text{ is a small amount.} \end{cases}$$

Sim

$$\mathbf{T}_S = \begin{bmatrix} s\mathbf{R} & \mathbf{t} \\ \mathbf{0}^T & 1 \end{bmatrix}$$

Affine

$$\mathbf{T}_A = \begin{bmatrix} \mathbf{A} & \mathbf{t} \\ \mathbf{0}^T & 1 \end{bmatrix}$$

Perspective

$$\mathbf{T}_P = \begin{bmatrix} \mathbf{A} & \mathbf{t} \\ \mathbf{a}^T & v \end{bmatrix}$$

Newton

$$\Delta\mathbf{x}^* = \arg\min\left(\mathbf{F}(\mathbf{x}) + \mathbf{J}(\mathbf{x})^T\Delta\mathbf{x} + \frac{1}{2}\Delta\mathbf{x}^T\mathbf{H}\Delta\mathbf{x}\right).$$

Gauss-Newton

$$\frac{1}{2}\|\mathbf{f}(\mathbf{x}) + \mathbf{J}(\mathbf{x})^T\Delta\mathbf{x}\|^2 = \frac{1}{2}(\mathbf{f}(\mathbf{x}) + \mathbf{J}(\mathbf{x})^T\Delta\mathbf{x})^T(\mathbf{f}(\mathbf{x}) + \mathbf{J}(\mathbf{x})^T\Delta\mathbf{x})$$
$$= \frac{1}{2}(\|\mathbf{f}(\mathbf{x})\|_2^2 + 2\mathbf{f}(\mathbf{x})\mathbf{J}(\mathbf{x})^T\Delta\mathbf{x} + \Delta\mathbf{x}^T\mathbf{J}(\mathbf{x})\mathbf{J}(\mathbf{x})^T\Delta\mathbf{x}).$$

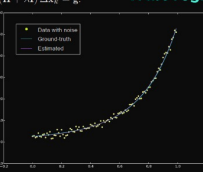
$$\underbrace{\mathbf{J}(\mathbf{x})\mathbf{J}^T(\mathbf{x})}_{\mathbf{H}(\mathbf{x})}\Delta\mathbf{x} = \underbrace{-\mathbf{J}(\mathbf{x})\mathbf{f}(\mathbf{x})}_{\mathbf{g}(\mathbf{x})}.$$

Levenberg-Marquatdt

$$\mathcal{L}(\Delta\mathbf{x}_k, \lambda) = \frac{1}{2}\left\|\mathbf{f}(\mathbf{x}_k) + \mathbf{J}(\mathbf{x}_k)^T\Delta\mathbf{x}_k\right\|^2 + \frac{\lambda}{2}\left(\|\mathbf{D}\Delta\mathbf{x}_k\|^2 - \mu\right),$$

Trust region

$$(\mathbf{H} + \lambda\mathbf{I})\Delta\mathbf{x}_k = \mathbf{g}.$$



Batch State Estimation

$$\begin{cases} \mathbf{x}_k = \mathbf{f}(\mathbf{x}_{k-1}, \mathbf{u}_k) + \mathbf{w}_k \\ \mathbf{z}_{k,j} = \mathbf{h}(\mathbf{y}_j, \mathbf{x}_k) + \mathbf{v}_{k,j} \end{cases}$$

$$P(\mathbf{x}, \mathbf{y}|\mathbf{z}, \mathbf{u}) = \frac{P(\mathbf{z}, \mathbf{u}|\mathbf{x}, \mathbf{y})P(\mathbf{x}, \mathbf{y})}{P(\mathbf{z}, \mathbf{u})} \propto \underbrace{P(\mathbf{z}, \mathbf{u}|\mathbf{x}, \mathbf{y})}_{\text{likehood}}\underbrace{P(\mathbf{x}, \mathbf{y})}_{\text{prior}}.$$

MAP Estimation

$$(\mathbf{x}, \mathbf{y})^*_{\text{MAP}} = \arg\max P(\mathbf{x}, \mathbf{y}|\mathbf{z}, \mathbf{u}) = \arg\max P(\mathbf{z}, \mathbf{u}|\mathbf{x}, \mathbf{y})P(\mathbf{x}, \mathbf{y}).$$

Under Gaussian

$$-\ln(P(\mathbf{x})) = \frac{1}{2}\ln\left((2\pi)^N\det(\boldsymbol{\Sigma})\right) + \frac{1}{2}(\mathbf{x} - \boldsymbol{\mu})^T\boldsymbol{\Sigma}^{-1}(\mathbf{x} - \boldsymbol{\mu}).$$

$$(\mathbf{x}_k, \mathbf{y}_j)^* = \arg\max\mathcal{N}(h(\mathbf{y}_j, \mathbf{x}_k), \mathbf{Q}_{k,j})$$
$$= \arg\min\left((\mathbf{z}_{k,j} - h(\mathbf{x}_k, \mathbf{y}_j))^T\mathbf{Q}_{k,j}^{-1}(\mathbf{z}_{k,j} - h(\mathbf{x}_k, \mathbf{y}_j))\right)$$

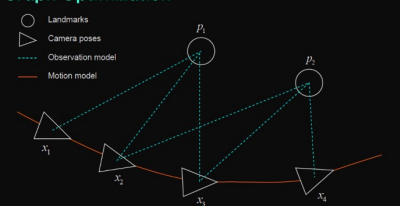
Batch Least Square

$$\mathbf{e}_{u,k} = \mathbf{x}_k - f(\mathbf{x}_{k-1}, \mathbf{u}_k)$$
$$\mathbf{e}_{z,j,k} = \mathbf{z}_{k,j} - h(\mathbf{x}_k, \mathbf{y}_j),$$
$$\min J(\mathbf{x}, \mathbf{y}) = \sum_k \mathbf{e}_{u,k}^T\mathbf{R}_k^{-1}\mathbf{e}_{u,k} + \sum_k\sum_j \mathbf{e}_{z,k,j}^T\mathbf{Q}_{k,j}^{-1}\mathbf{e}_{z,k,j}.$$

Kalman Filter

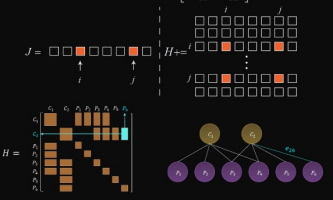
1. *Predict*:
$$\bar{\mathbf{x}}_k = \mathbf{A}_k\hat{\mathbf{x}}_{k-1} + \mathbf{u}_k, \quad \bar{\mathbf{P}}_k = \mathbf{A}_k\mathbf{P}_{k-1}\mathbf{A}_k^T + \mathbf{R}. \tag{8.24}$$
2. *Update*: Calculate $\mathbf{K}$, which is the Kalman gain:
$$\mathbf{K} = \bar{\mathbf{P}}_k\mathbf{C}_k^T(\mathbf{C}_k\bar{\mathbf{P}}_k\mathbf{C}_k^T + \mathbf{Q}_k)^{-1}, \tag{8.25}$$
and the posterior:
$$\hat{\mathbf{x}}_k = \bar{\mathbf{x}}_k + \mathbf{K}(\mathbf{z}_k - \mathbf{C}_k\bar{\mathbf{x}}_k)$$
$$\hat{\mathbf{P}}_k = (\mathbf{I} - \mathbf{K}\mathbf{C}_k)\bar{\mathbf{P}}_k. \tag{8.26}$$

Graph Optimization



Sparse BA

$$\mathbf{H} = \begin{bmatrix} \mathbf{H}_{11} & \mathbf{H}_{12} \\ \mathbf{H}_{21} & \mathbf{H}_{22} \end{bmatrix}.$$



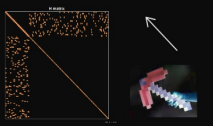
Schur-trick

$$\begin{bmatrix} \mathbf{I} & -\mathbf{E}\mathbf{C}^{-1} \\ \mathbf{0} & \mathbf{I} \end{bmatrix}\begin{bmatrix} \mathbf{B} & \mathbf{E} \\ \mathbf{E}^T & \mathbf{C} \end{bmatrix}\begin{bmatrix} \Delta\mathbf{x}_c \\ \Delta\mathbf{x}_p \end{bmatrix} = \begin{bmatrix} \mathbf{I} & -\mathbf{E}\mathbf{C}^{-1} \\ \mathbf{0} & \mathbf{I} \end{bmatrix}\begin{bmatrix} \mathbf{v} \\ \mathbf{w} \end{bmatrix}.$$

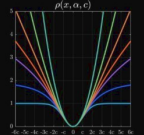
$$\begin{bmatrix} \mathbf{B} - \mathbf{E}\mathbf{C}^{-1}\mathbf{E}^T & \mathbf{0} \\ \mathbf{E}^T & \mathbf{C} \end{bmatrix}\begin{bmatrix} \Delta\mathbf{x}_c \\ \Delta\mathbf{x}_p \end{bmatrix} = \begin{bmatrix} \mathbf{v} - \mathbf{E}\mathbf{C}^{-1}\mathbf{w} \\ \mathbf{w} \end{bmatrix}.$$

$$[\mathbf{B} - \mathbf{E}\mathbf{C}^{-1}\mathbf{E}^T]\Delta\mathbf{x}_c = \mathbf{v} - \mathbf{E}\mathbf{C}^{-1}\mathbf{w}.$$

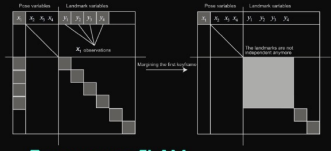
Arrow-like Hessian



Robust kernel



Fill-in in Marginalization

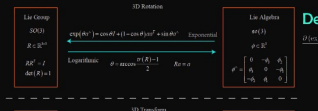


Open-source SLAM

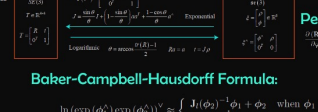
Name	Sensors*	URL
MonoSLAM	M	https://github.com/hanmekim/SceneLib2
PTAM	M	http://www.robots.ox.ac.uk/~gk/PTAM/
ORB-SLAM	M/S/R	http://webdiis.unizar.es/~raulmur/orbslam/
LSD-SLAM	M	http://vision.in.tum.de/research/vslam/lsdslam
SVO	M	https://github.com/uzh-rpg/rpg_svo
DTAM	R	https://github.com/anuranbaka/OpenDTAM
DVO	R	https://github.com/tum-vision/dvo_slam
DSO	M	https://github.com/JakobEngel/dso
VINS	M+IMU	https://github.com/HKUST-Aerial-Robotics/VINS-Mono
RTAB-MAP	S/R	https://github.com/introlab/rtabmap
RGBD-SLAM-V2	R	https://github.com/felixendres/rgbdslam_v2
Elastic Fusion	R	https://github.com/mp3guy/ElasticFusion
Hector SLAM	L	http://wiki.ros.org/hector_slam
GMapping	L	http://wiki.ros.org/gmapping
OKVIS	M/S+IMU	https://github.com/ethz-asl/okvis
ROVIO	M+IMU	https://github.com/ethz-asl/rovio

* M=Monocular, S=Stereo, R=RGB-D, L=Lidar.

Derivative Model:



Perturbation Model



Left/Right Jacobians

$$\mathbf{J}_l = \mathbf{J} = \frac{\sin\theta}{\theta}\mathbf{I} + \left(1 - \frac{\sin\theta}{\theta}\right)\mathbf{a}\mathbf{a}^T + \frac{1 - \cos\theta}{\theta}\mathbf{a}^\wedge.$$

$$\mathbf{J}_r(\boldsymbol{\phi}) = \mathbf{J}_l(-\boldsymbol{\phi}).$$

Contents

- Lecture 1. Introduction to 3D Geometry
- Lecture 2. Visual Odometry and Backend Optimization

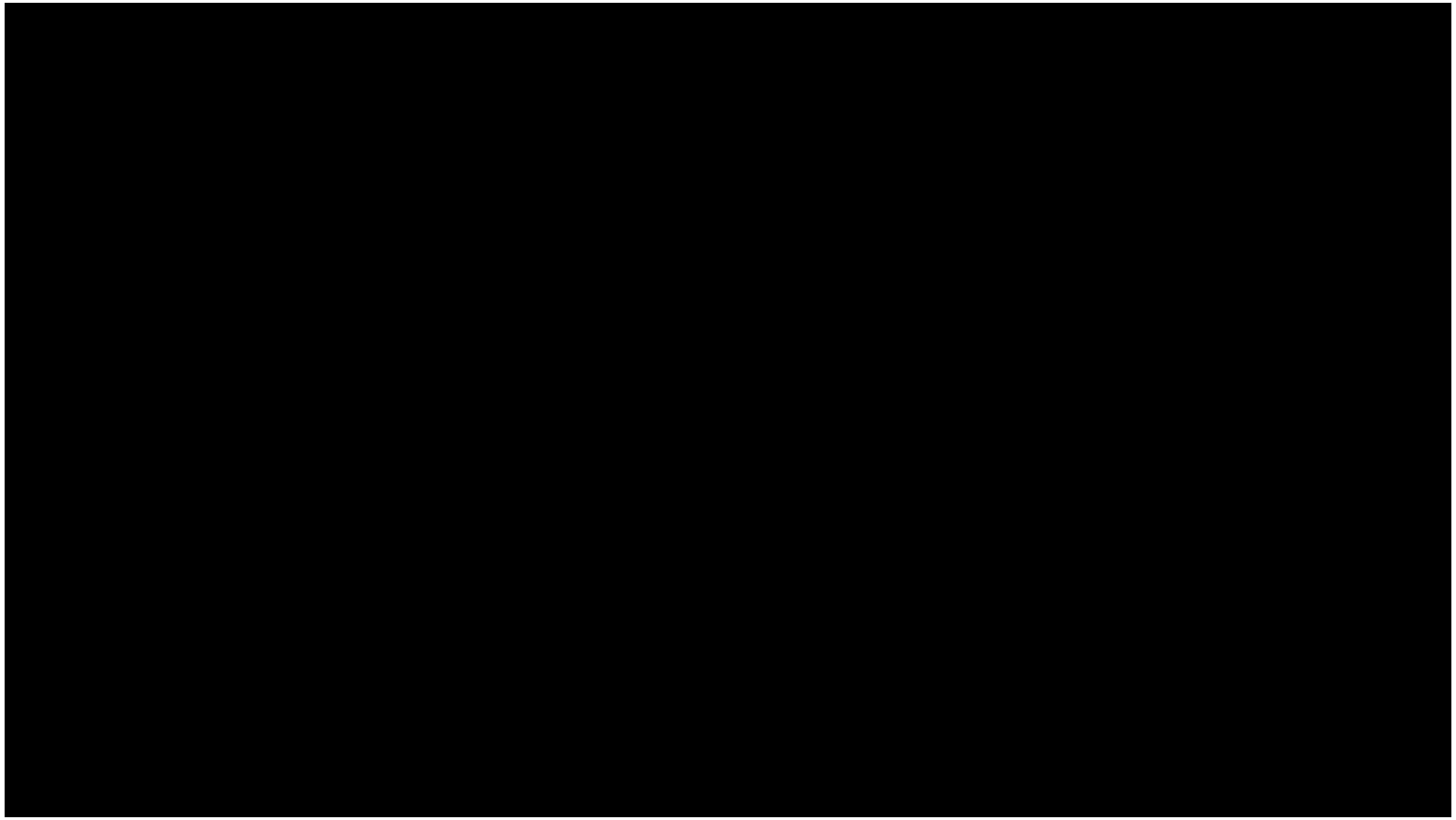
Lecture 1. Introduction to 3D Geometry

- Framework and mathematic form of a SLAM problem
- 3D geometry
- State Estimation, Nonlinear Optimization and Least Square

Framework and mathematic form of a SLAM problem

Preliminary Knowledge

- General overview of computer vision tasks

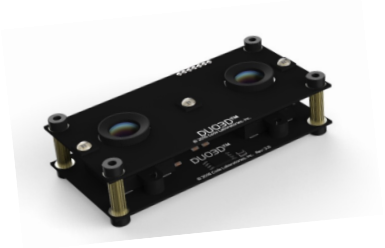


Preliminary Knowledge

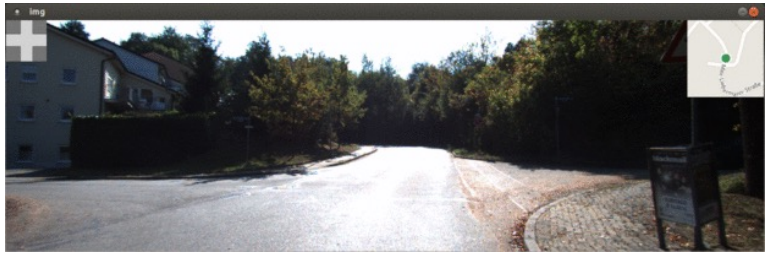
- Computer vision











Object detection
Object recognition
Object tracking
Segmentation
…
SLAM

Real world cameras

Image and video sequences

CV tasks

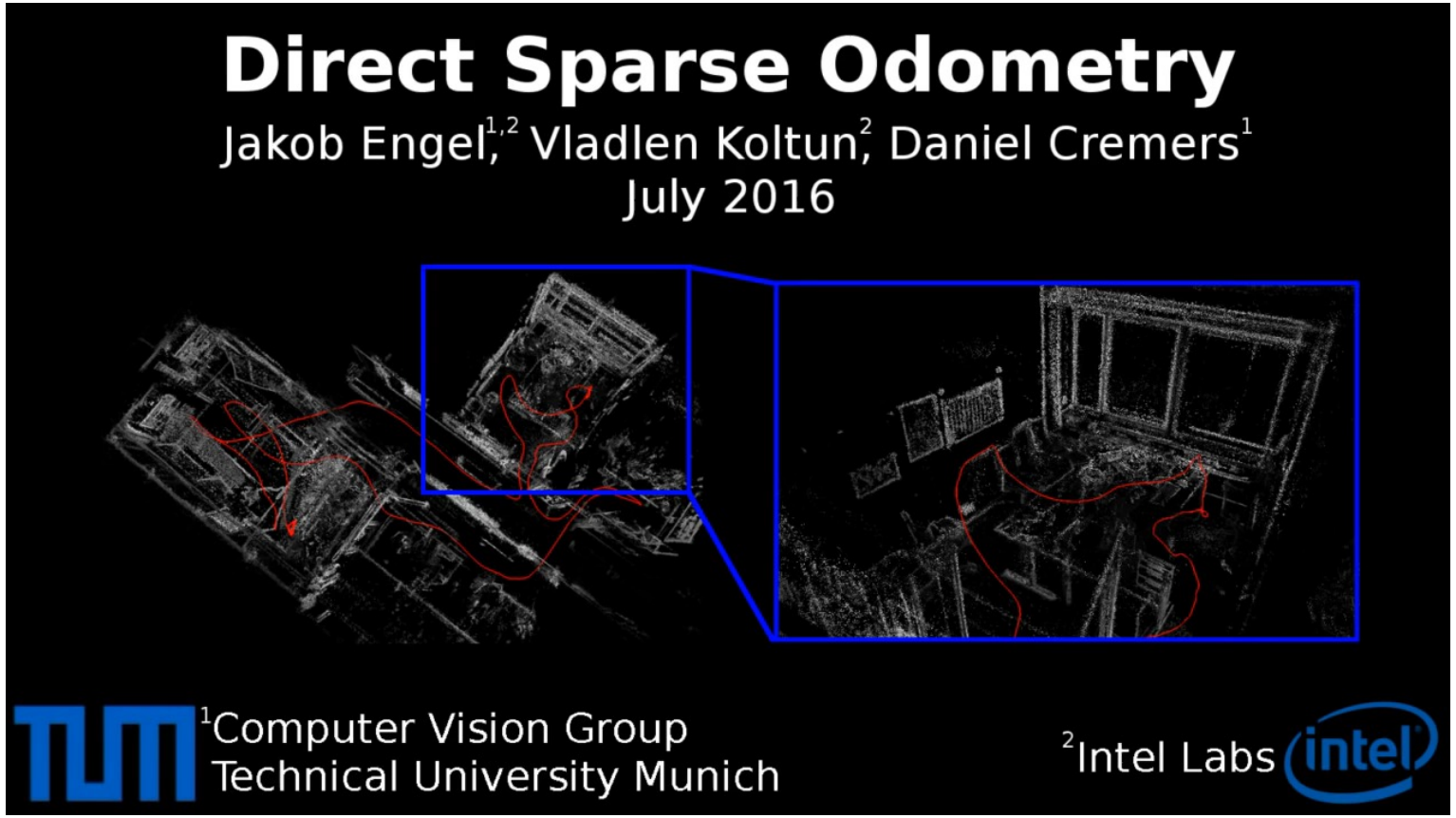
Preliminary Knowledge

- What is SLAM? - Simultaneous Localization And Mapping



Preliminary Knowledge

- Dense and semi-dense reconstruction



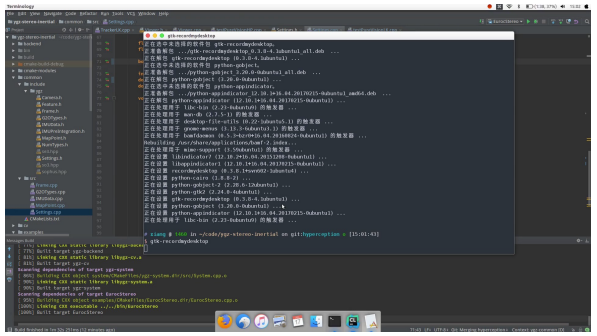
Preliminary Knowledge

Dense reconstruction with RGB-D cameras

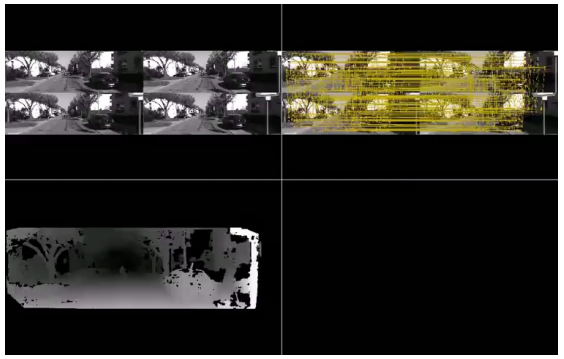


Preliminary Knowledge

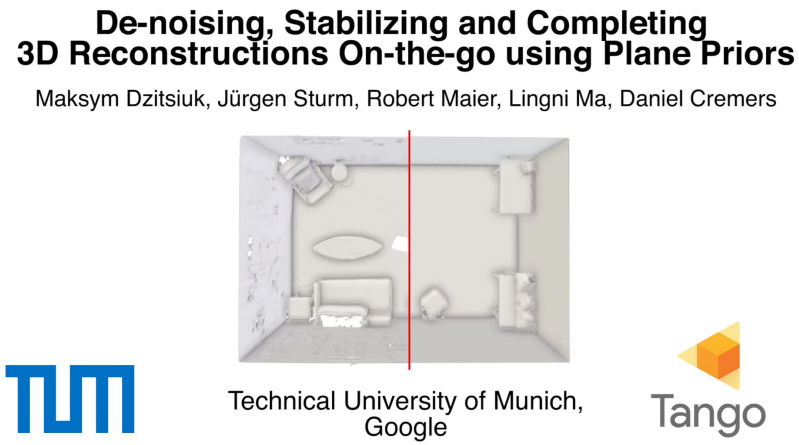
- Applications of SLAM



Hand-held devices



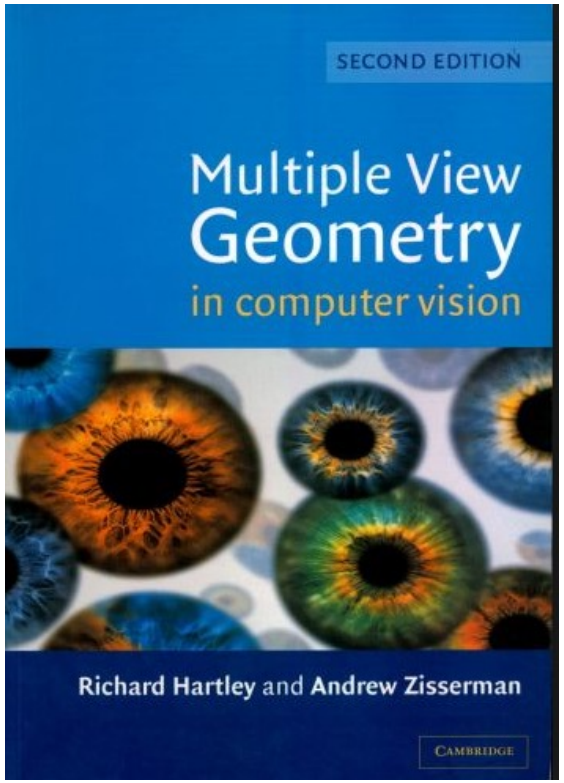
Autonomous driving

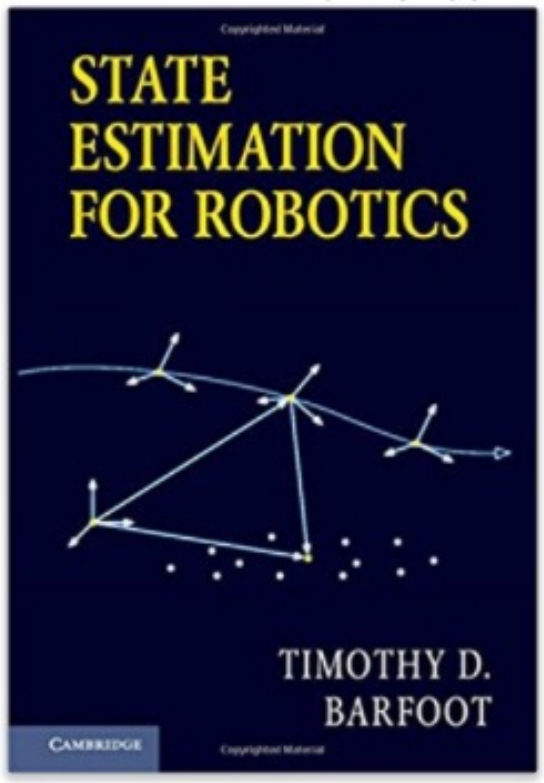


Augmented reality/VR

Preliminary Knowledge

- Related books





Harley and Zisserman, Multiple view geometry in computer vision

Tim Barfoot, State estimation for robotics

Framework of SLAM

- SLAM problem
  - Fundamental problems in intelligent robots
  - Where am I?
    -Localization
  - What is around me?
    -Mapping

- Chicken and egg problem
  - Localization needs accurate map
  - Mapping needs accurate localization



Framework of SLAM

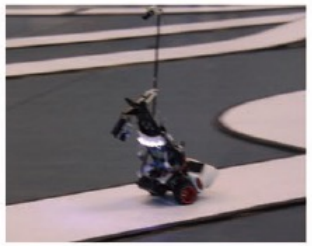
- How to do SLAM? -Sensors
- Sensor is the way to measure the outside environment
- Interoseptive sensors: accelerometer, gyroscope …
- Exteroceptive sensors: camera, laser rangefinder, GPS …



(a)



(b)



(c)



(d)



(e)



(f)

Some sensors must be placed in a cooperative environment, other can be directly equipped in the robot itself

Framework of SLAM

- Visual SLAM -SLAM (mainly) by cameras
- Cameras
  - Monocular
  - Stereo
  - RGB-D
  - Omnidirectional, Event camera, etc
- Cameras
  - Cheap, rich information
  - Record 2D projected image of the 3D world
  - The 3D-2D projection throws away one dimension: distance



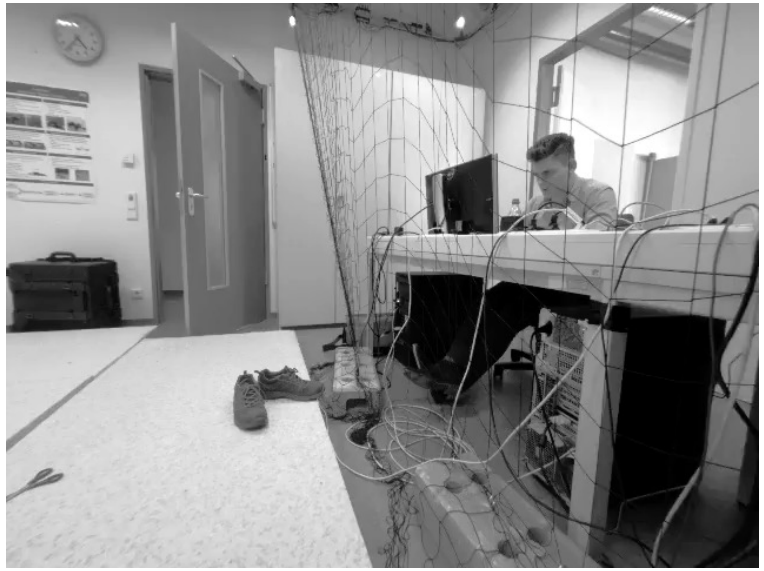
Monocular camera



RGB-D (depth) camera



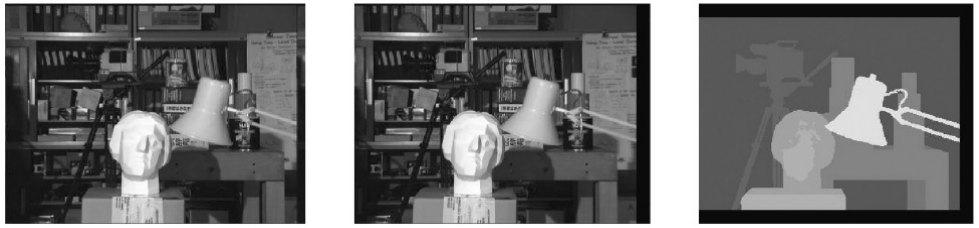
Stereo camera



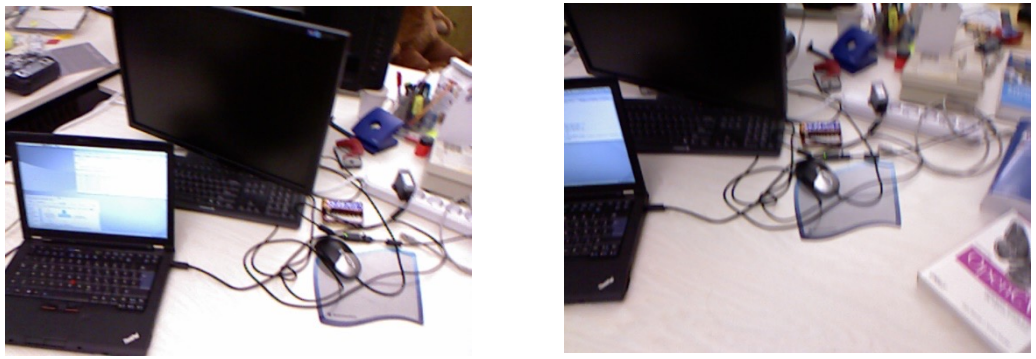
Framework of SLAM

- Various kinds of cameras:
- Monocular: image only, need other methods to estimate the depth
- Stereo: disparity to depth
- RGB-D: physical depth measurements





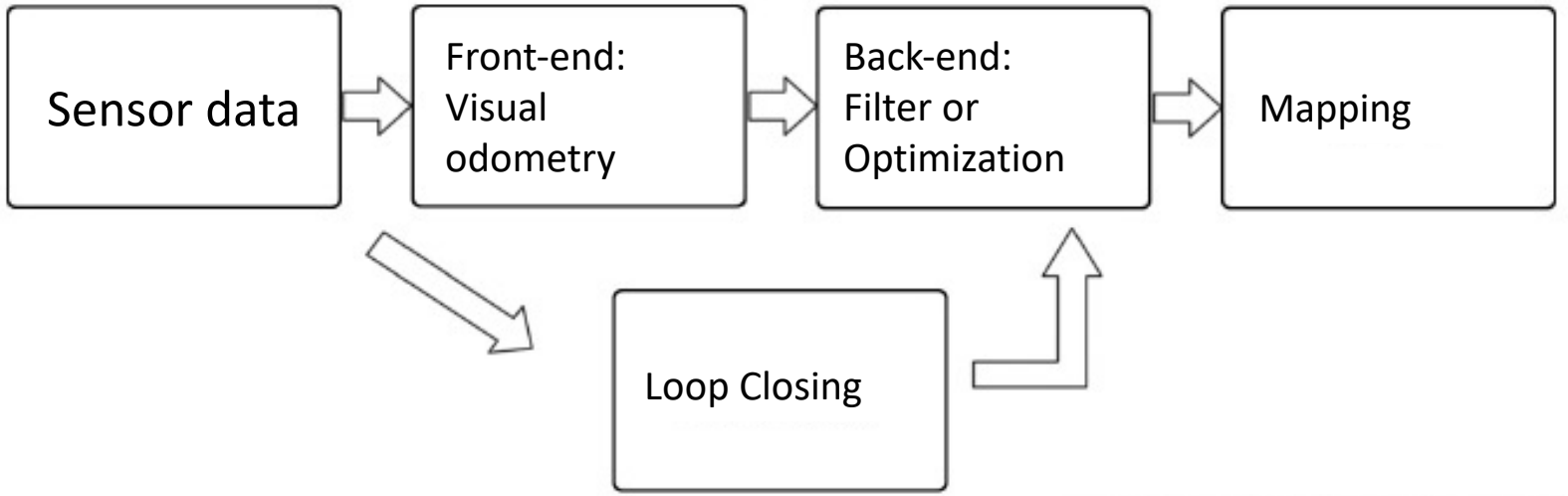
Stereo vision estimates the depth from disparity

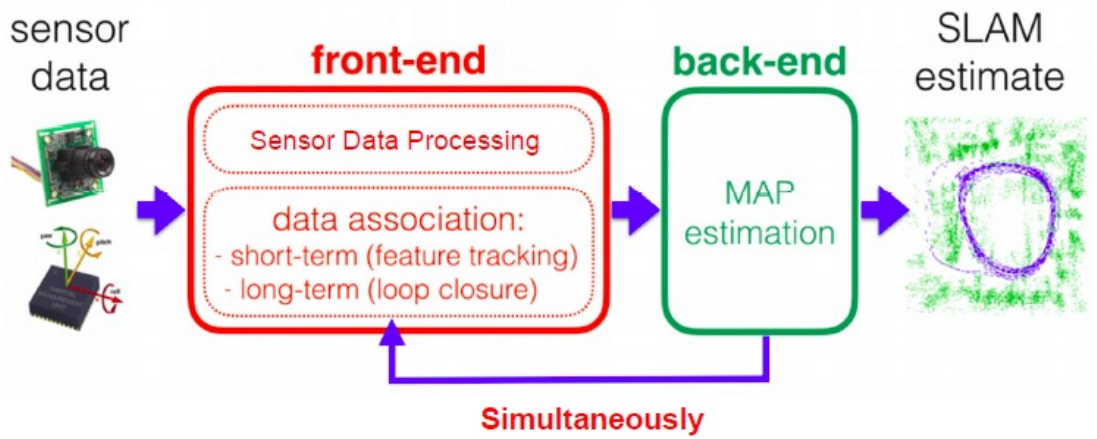


Moving stereo: disparity can be estimated in the motion

Ambiguity in mono vision: small + close or large + far away?

Framework of SLAM

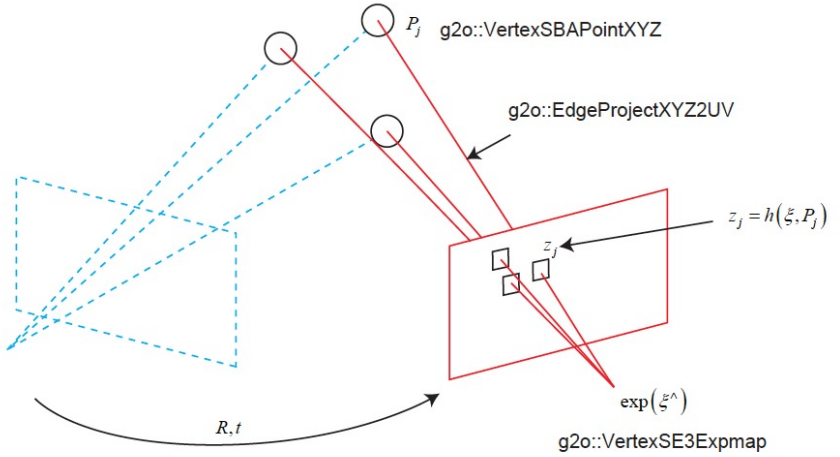




Framework of SLAM

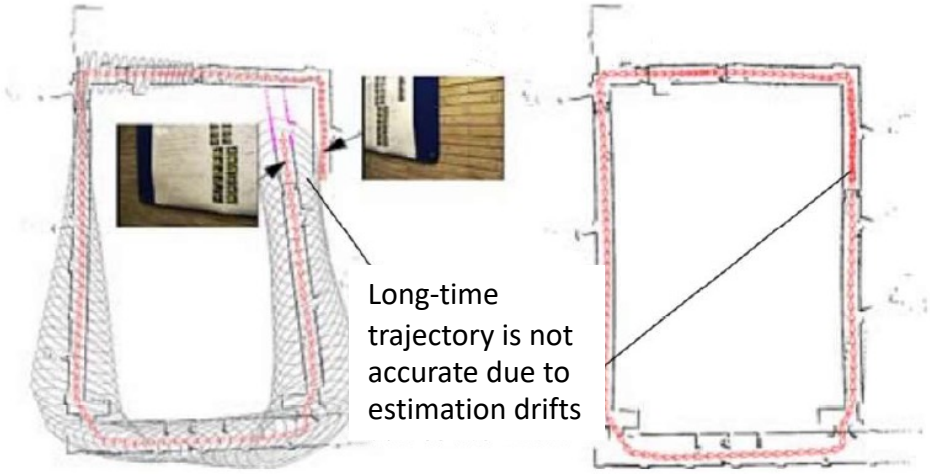
- Visual odometry
  - Motion estimation between adjacent frames
  - Simplest: two-view geometry
- Method
  - Feature method
  - Direct method
- Backend
  - Long-term trajectory and map estimation
  - MAP: Maximum of a Posteri
  - Filters/Graph Optimization

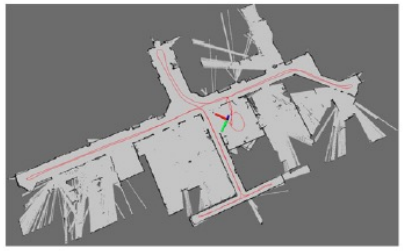




Framework of SLAM

- Loop closing
  - Correct the drift in estimation
  - Loop detection and correction
- Mapping
  - Generate globally consistent map for navigation/planning/communication/visualization etc
  - Grid/topological/hybrid maps
  - Point cloud/Mesh/TSDF …

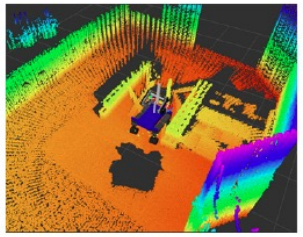




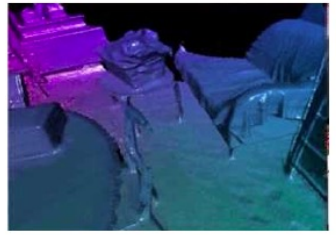
2D grid map



2D topological map



Point cloud maps



TSDF models

Framework of SLAM

- Mathematical representation of visual SLAM
- Assume a camera is moving in 3D space
  - But measurements are taken at discrete times:

$$\begin{cases} \boldsymbol{x}_k = f\left(\boldsymbol{x}_{k-1}, \boldsymbol{u}_k, \boldsymbol{w}_k\right) \\ \boldsymbol{z}_{k,j} = h\left(\boldsymbol{y}_j, \boldsymbol{x}_k, \boldsymbol{v}_{k,j}\right) \end{cases}$$

Motion model

Observation model

Non-linear form

$$\begin{cases} x_k = A_k x_{k-1} + B_k u_k + w_k \\ z_{k,j} = C_j y_j + D_k x_k + v_{k,j} \end{cases}$$

linear form

Framework of SLAM

- Questions:

$$\begin{cases} \boldsymbol{x}_k = f\left(\boldsymbol{x}_{k-1}, \boldsymbol{u}_k, \boldsymbol{w}_k\right) & \text{Motion model} \\ \boldsymbol{z}_{k,j} = h\left(\boldsymbol{y}_j, \boldsymbol{x}_k, \boldsymbol{v}_{k,j}\right) & \text{Observation model} \end{cases}.$$

- How to represent state variables?
  - 3D geometry, Lie group and Lie algebra
- Exact form of motion/observation model?
  - Camera intrinsic and extrinsics
- How to estimate the state given measurement data?
  - State estimation problem
  - Filters and optimization

3D Geometry

课程回放：https://www.shenlanxueyuan.com/course/386

3D Geometry

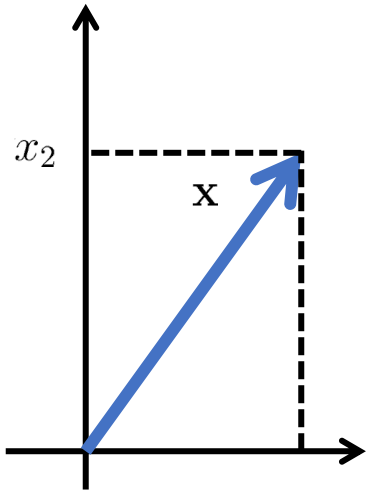
- Point and Coordinate system
- 2D: (x,y) and angle
- 3D?

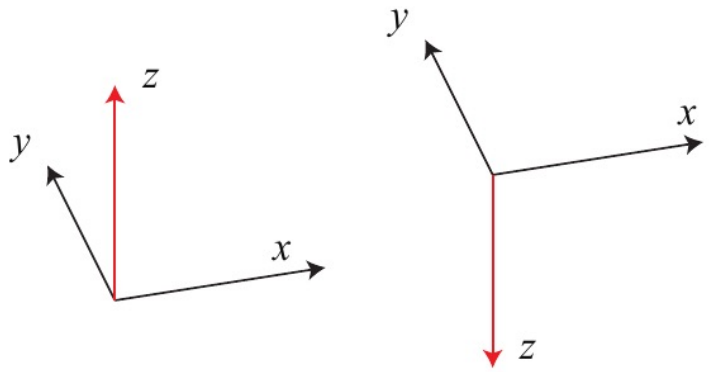
3D Geometry

- Vector and its coordinates

$$\mathbf{x} = \begin{pmatrix} x_1 \\ x_2 \\ \vdots \\ x_n \end{pmatrix} \in \mathbb{R}^n$$

- Vectors represent points in an n-dimensional space



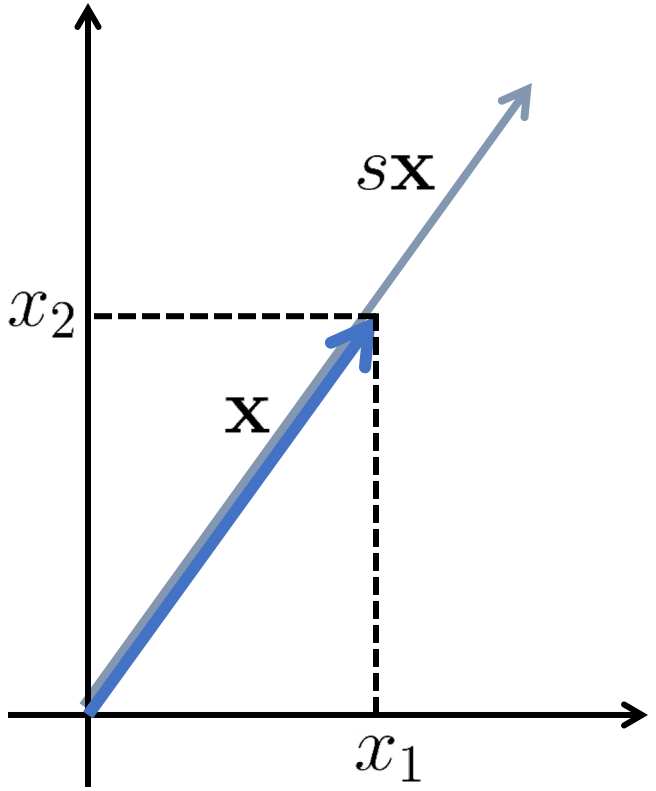


Right handed

Left handed

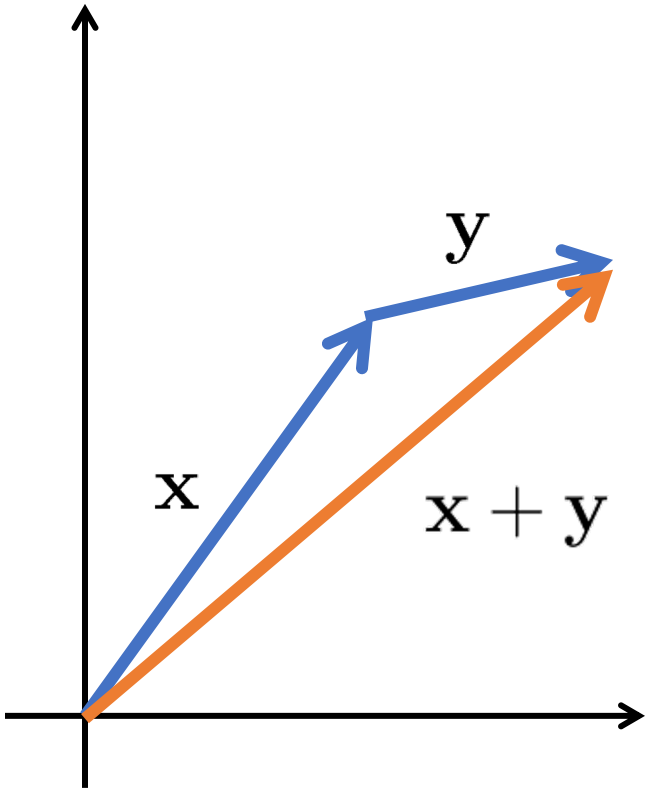
3D Geometry

- **Scalar multiplication**
- Addition/subtraction
- Length
- Normalized vector
- Dot product
- Cross product



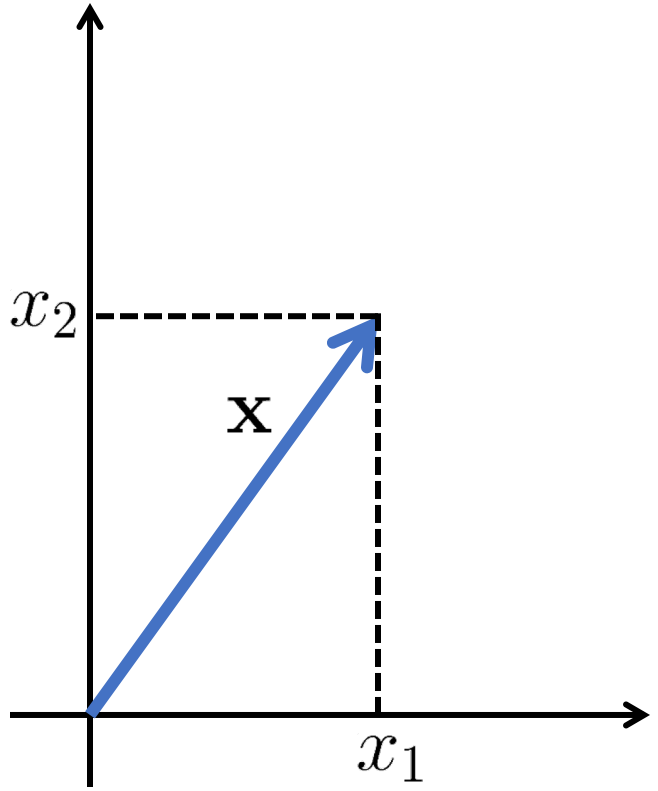
3D Geometry

- Scalar multiplication
- **Addition/subtraction**
- Length
- Normalized vector
- Dot product
- Cross product



3D Geometry

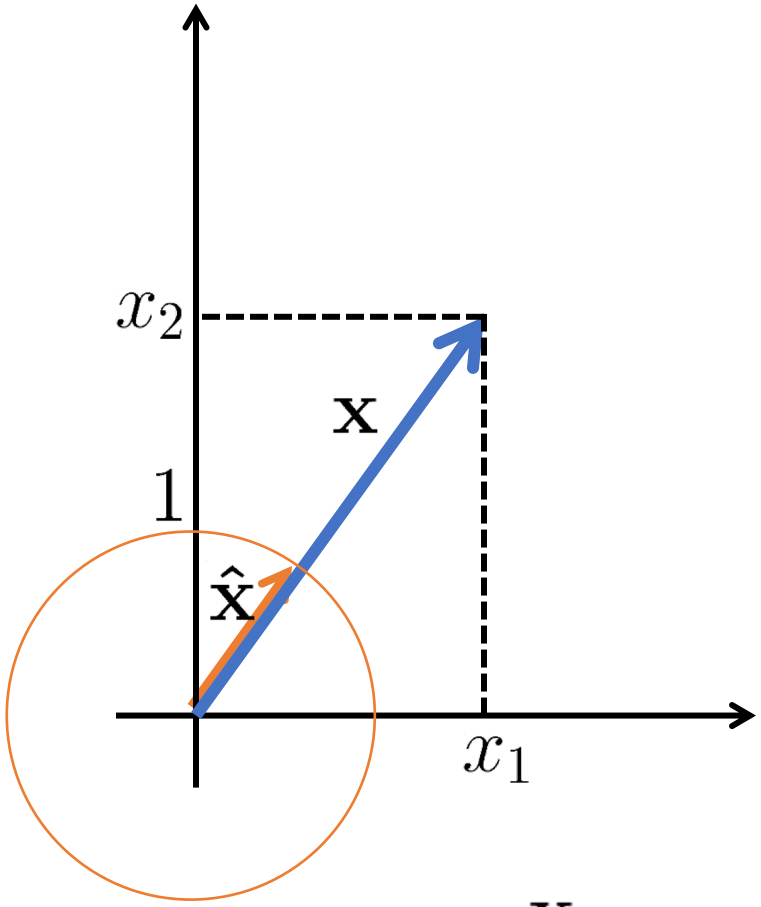
- Scalar multiplication
- Addition/subtraction
- **Length**
- Normalized vector
- Dot product
- Cross product



$$\|x\|_2 = \|x\| = \sqrt{x_1^2 + x_2^2 + \dots}$$

3D Geometry

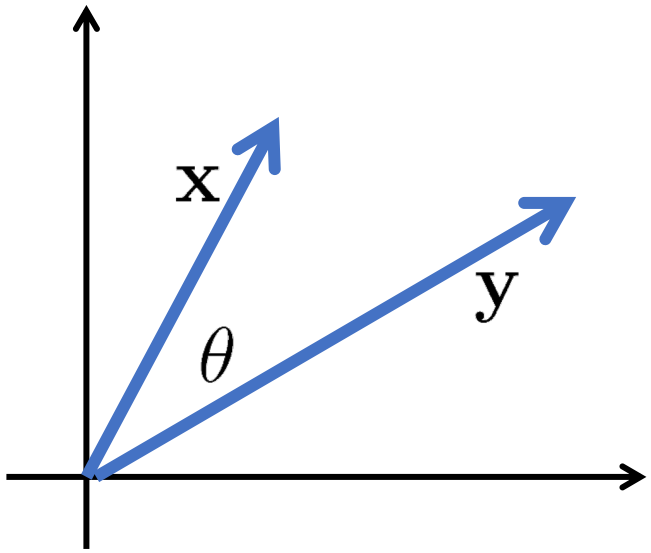
- Scalar multiplication
- Addition/subtraction
- Length
- **Normalized vector**
- Dot product
- Cross product



$$\hat{\mathbf{x}} = \frac{\mathbf{x}}{\|\mathbf{x}\|}$$

3D Geometry

- Scalar multiplication
- Addition/subtraction
- Length
- Normalized vector
- **Dot product**
- Cross product



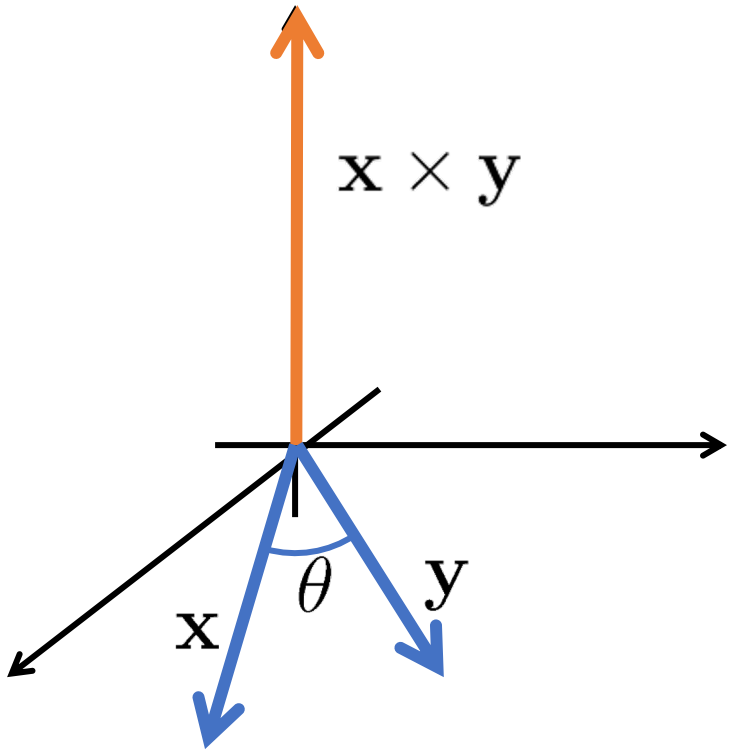
$$\mathbf{x} \cdot \mathbf{y} = \|\mathbf{x}\| \|\mathbf{y}\| \cos\theta$$

$\mathbf{x}, \mathbf{y}$ are orthogonal if $\mathbf{x} \cdot \mathbf{y} = 0$

$\{\mathbf{x}_1, \mathbf{x}_2, \ldots\}$ is lin. dependent from $\mathbf{y}$ if $\mathbf{y} = \sum_i k_i \mathbf{x}_i$

3D Geometry

- Scalar multiplication
- Addition/subtraction
- Length
- Normalized vector
- Dot product
- **Cross product**



$$\mathbf{x} \times \mathbf{y} = \|\mathbf{x}\| \|\mathbf{y}\| \sin(\theta)\mathbf{n}$$

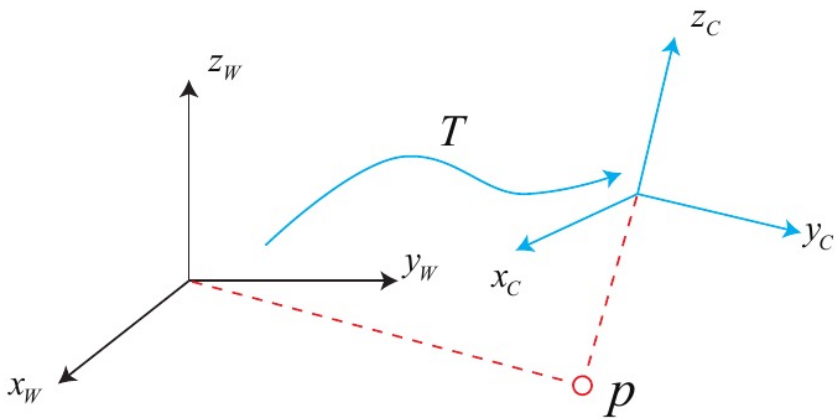
3D Geometry

- Cross product:

$$\boldsymbol{a} \times \boldsymbol{b} = \begin{bmatrix} \boldsymbol{i} & \boldsymbol{j} & \boldsymbol{k} \\ a_1 & a_2 & a_3 \\ b_1 & b_2 & b_3 \end{bmatrix} = \begin{bmatrix} a_2 b_3 - a_3 b_2 \\ a_3 b_1 - a_1 b_3 \\ a_1 b_2 - a_2 b_1 \end{bmatrix} = \begin{bmatrix} 0 & -a_3 & a_2 \\ a_3 & 0 & -a_1 \\ -a_2 & a_1 & 0 \end{bmatrix} \boldsymbol{b} \triangleq \boldsymbol{a}^{\wedge} \boldsymbol{b}.$$

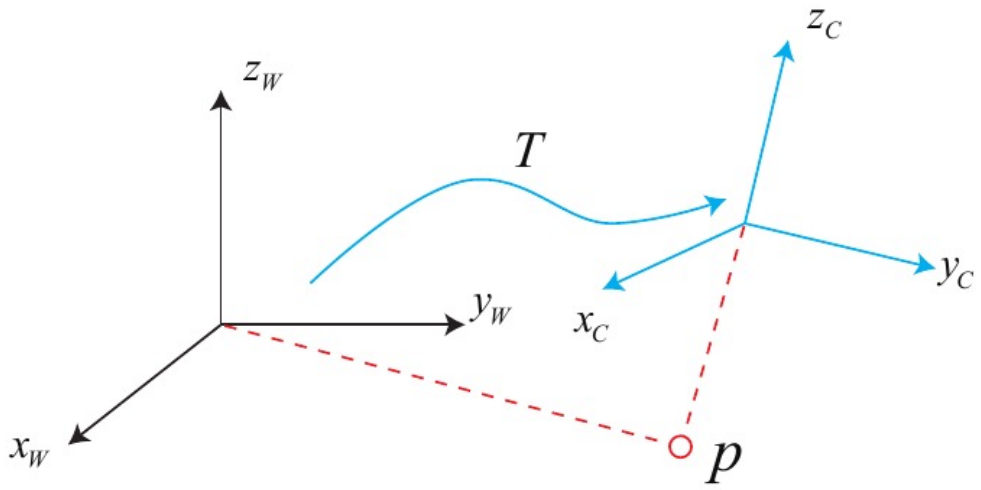
Skew-symmetric operator

- Questions
  - Compute the coordinates in different systems?
- In SLAM:
  - Fixed world frame
  - Moving camera frame
  - Other sensor frames



3D Geometry

- 3D rigid body motion can be described with rotation and translation



- Translation is just a vector addition
- How to represent rotations?

3D Geometry

- Rotation
  - Consider coordinate system (e_1, e_2, e_3)otated and becomes $(e_1^{'}, e_2^{'}, e_3^{'})$
  - Vector a is fixed, then how are its coordinates changed?

$$[e_1, e_2, e_3] \begin{bmatrix} a_1 \\ a_2 \\ a_3 \end{bmatrix} = \left[e_1^{'}, e_2^{'}, e_3^{'}\right] \begin{bmatrix} a_1^{'} \\ a_2^{'} \\ a_3^{'} \end{bmatrix}.$$

  - Left multiplied by $[e_1^T, e_2^T, e_3^T]^T$

$$\begin{bmatrix} a_1 \\ a_2 \\ a_3 \end{bmatrix} = \begin{bmatrix} e_1^T e_1^{'} & e_1^T e_2^{'} & e_1^T e_3^{'} \\ e_2^T e_1^{'} & e_2^T e_2^{'} & e_2^T e_3^{'} \\ e_3^T e_1^{'} & e_3^T e_2^{'} & e_3^T e_3^{'} \end{bmatrix} \begin{bmatrix} a_1^{'} \\ a_2^{'} \\ a_3^{'} \end{bmatrix} \triangleq \boldsymbol{R}\boldsymbol{a}^{'}.$$

3D Geometry

- R is rotation matrix, which satisfies:
  - R is orthogonal
  - Det(R) = +1 (if Det(R)=-1 then it's improper rotation)

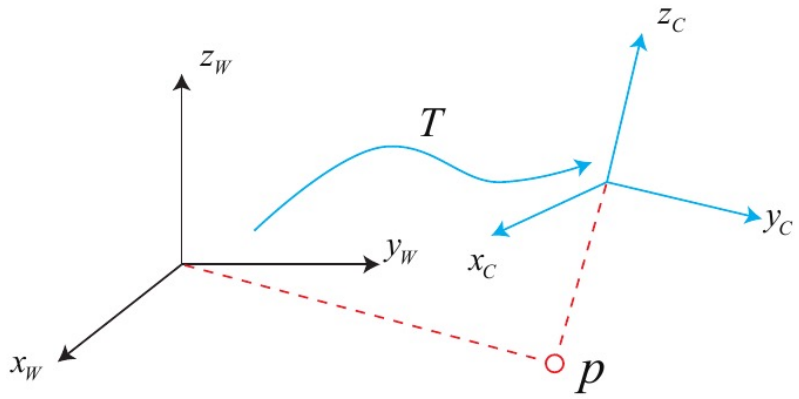
- Special orthogonal group: $SO(n) = \{\boldsymbol{R} \in \mathbb{R}^{n\times n} | \boldsymbol{R}\boldsymbol{R}^T = \boldsymbol{I}, \det(\boldsymbol{R}) = 1\}.$

- Rotation from frame 2 to 1 can be written as:

$a_1 = R_{12}a_2$ and vise vesa: $a_2 = R_{21}a_1$

$$\boxed{R_{21} = R_{12}^{-1} = R_{12}^T}$$

3D Geometry



- Rotation plus translation:

$$\boldsymbol{a}' = \boldsymbol{R}\boldsymbol{a} + \boldsymbol{t}.$$

- Compounding rotation and translation:
  - $\boldsymbol{b} = \boldsymbol{R}_1\boldsymbol{a} + \boldsymbol{t}_1, \quad \boldsymbol{c} = \boldsymbol{R}_2\boldsymbol{b} + \boldsymbol{t}_2.$ ⟹ $\boldsymbol{c} = \boldsymbol{R}_2\left(\boldsymbol{R}_1\boldsymbol{a} + \boldsymbol{t}_1\right) + \boldsymbol{t}_2.$
- Homogeneous form:

$$\begin{bmatrix} \boldsymbol{a}' \\ 1 \end{bmatrix} = \begin{bmatrix} \boldsymbol{R} & \boldsymbol{t} \\ \boldsymbol{0}^T & 1 \end{bmatrix} \begin{bmatrix} \boldsymbol{a} \\ 1 \end{bmatrix} \triangleq \boldsymbol{T} \begin{bmatrix} \boldsymbol{a} \\ 1 \end{bmatrix}.$$

$$\tilde{\boldsymbol{b}} = \boldsymbol{T}_1\tilde{\boldsymbol{a}},\ \tilde{\boldsymbol{c}} = \boldsymbol{T}_2\tilde{\boldsymbol{b}} \quad \Rightarrow \tilde{\boldsymbol{c}} = \boldsymbol{T}_2\boldsymbol{T}_1\tilde{\boldsymbol{a}}.$$

Inverse: $$\boldsymbol{T}^{-1} = \begin{bmatrix} \boldsymbol{R}^T & -\boldsymbol{R}^T t \\ \boldsymbol{0}^T & 1 \end{bmatrix}.$$

3D Geometry

- Homogenous coordinates:

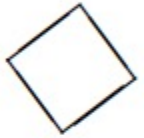
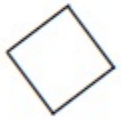
$$\tilde{a} = \begin{bmatrix} a \\ 1 \end{bmatrix} \qquad \tilde{a} = \begin{bmatrix} a \\ 1 \end{bmatrix} = k \begin{bmatrix} a \\ 1 \end{bmatrix}$$

Still keeps equal when multiplying any non-zero factors

- Transform matrix forms Special Euclidean Group

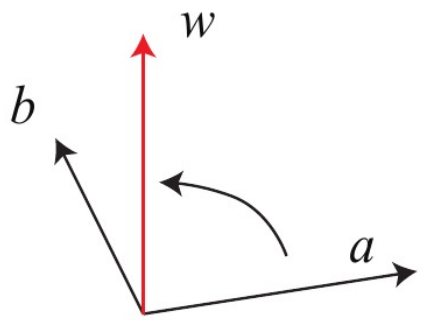
$$SE(3) = \left\{ \boldsymbol{T} = \begin{bmatrix} \boldsymbol{R} & \boldsymbol{t} \\ \boldsymbol{0}^T & 1 \end{bmatrix} \in \mathbb{R}^{4\times 4} | \boldsymbol{R} \in SO(3), \boldsymbol{t} \in \mathbb{R}^3 \right\}.$$

3D Geometry

Transformation	Matrix	# DoF	Preserves	Icon
translation	$\left[\begin{array}{c\|c} \boldsymbol{I} & \boldsymbol{t} \end{array}\right]_{3\times4}$	3	orientation	
rigid (Euclidean)	$\left[\begin{array}{c\|c} \boldsymbol{R} & \boldsymbol{t} \end{array}\right]_{3\times4}$	6	lengths	
similarity	$\left[\begin{array}{c\|c} s\boldsymbol{R} & \boldsymbol{t} \end{array}\right]_{3\times4}$	7	angles	
affine	$\left[\begin{array}{c} \boldsymbol{A} \end{array}\right]_{3\times4}$	12	parallelism	
projective	$\left[\begin{array}{c} \tilde{\boldsymbol{H}} \end{array}\right]_{4\times4}$	15	straight lines	

3D Geometry

- Alternative rotation representations
  - Rotation vectors
  - Euler angles
  - Quaternions



Rotation vectors
Only three parameters

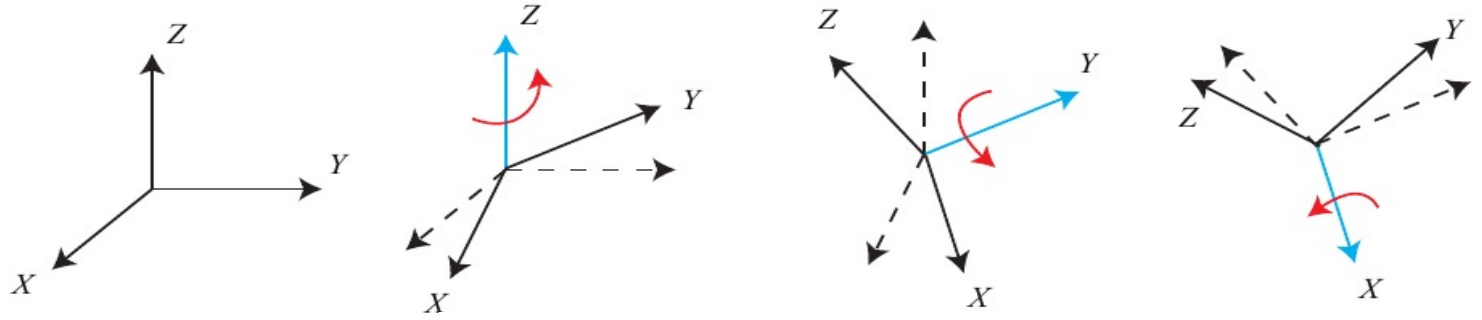
- Rotation vectors
  - Angle + axis: θn
  - Rotation angle θ
  - Rotation axis n
- Rotation vector to rotation matrix: Rodrigues' formula

$$\boldsymbol{R} = \cos\theta \boldsymbol{I} + (1 - \cos\theta)\,\boldsymbol{n}\boldsymbol{n}^T + \sin\theta \boldsymbol{n}^\wedge.$$

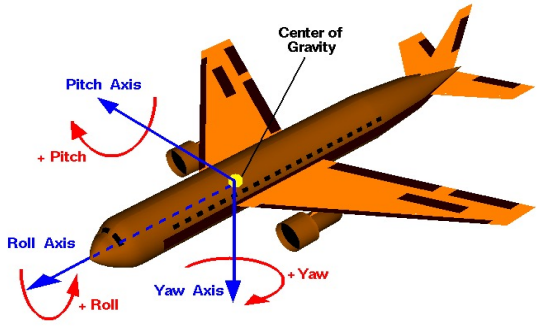
- Inverse: $\theta = \arccos\left(\frac{\mathrm{tr}(\boldsymbol{R}) - 1}{2}\right).$ $\boldsymbol{R}\boldsymbol{n} = \boldsymbol{n}.$

3D Geometry

- Euler angles
  - Any rotation can be decomposed into three principal rotations



Original First Second Third



  - However the order of axis can be defined very differently:
  - Roll-pitch-yaw (in navigation) Spin-nutation-precession in mechanics

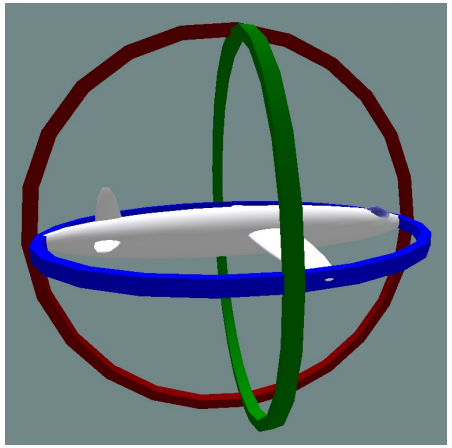
XYZ order

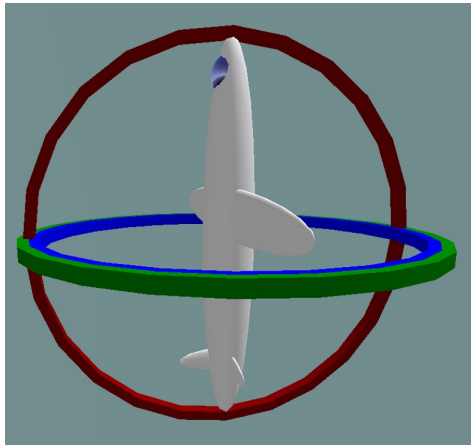
3-1-3 order

3D Geometry

- Gimbal lock
  - Singularity always exist if we want to use 3 parameters to describe rotation
  - Degree-of-Freedom is reduced in singular case
  - In yaw-pitch-roll order, when pitch=90 degrees

normal





singular

3D Geometry

- Quaternions
  - In 2D case, we can use (unit) complex numbers to denote rotations

$$z = x + iy = \rho e^{i\theta}$$

Multiply i to rotate 90 degrees

  - How about 3D case?
- (Unit) Quaternions
  - Extended from complex numbers
  - Three imaginary and one real part:
  - The imaginary parts satisfy:

$$\boldsymbol{q} = q_0 + q_1 i + q_2 j + q_3 k,$$

$$\begin{cases} i^2 = j^2 = k^2 = -1 \\ ij = k, ji = -k \\ jk = i, kj = -i \\ ki = j, ik = -j \end{cases}.$$

i,j,k look like complex numbers when multiplying with themselves
And look like cross product when multiply with others

3D Geometry

- Quaternions

$$\boldsymbol{q} = q_0 + q_1 i + q_2 j + q_3 k,$$

$$\boldsymbol{q} = [s, \boldsymbol{v}], \quad s = q_0 \in \mathbb{R}, \boldsymbol{v} = [q_1, q_2, q_3]^T \in \mathbb{R}^3,$$

- Operations

$$\boldsymbol{q}_a \pm \boldsymbol{q}_b = [s_a \pm s_b, \boldsymbol{v}_a \pm \boldsymbol{v}_b].$$

$$\begin{aligned}\boldsymbol{q}_a \boldsymbol{q}_b = \; & s_a s_b - x_a x_b - y_a y_b - z_a z_b \\ & + (s_a x_b + x_a s_b + y_a z_b - z_a y_b) i \\ & + (s_a y_b - x_a z_b + y_a s_b + z_a x_b) j \\ & + (s_a z_b + x_a y_b - y_b x_a + z_a s_b) k.\end{aligned}$$

$$\boldsymbol{q}_a \boldsymbol{q}_b = \left[s_a s_b - \boldsymbol{v}_a^T \boldsymbol{v}_b, s_a \boldsymbol{v}_b + s_b \boldsymbol{v}_a + \boldsymbol{v}_a \times \boldsymbol{v}_b\right].$$

$$\boldsymbol{q}_a^* = s_a - x_a i - y_a j - z_a k = [s_a, -\boldsymbol{v}_a].$$

$$\|\boldsymbol{q}_a\| = \sqrt{s_a^2 + x_a^2 + y_a^2 + z_a^2}.$$

$$\boldsymbol{q}^{-1} = \boldsymbol{q}^* / \|\boldsymbol{q}\|^2.$$

$$k\boldsymbol{q} = [ks, k\boldsymbol{v}].$$

$$\boldsymbol{q}_a \cdot \boldsymbol{q}_b = s_a s_b + x_a x_b i + y_a y_b j + z_a z_b k.$$

3D Geometry

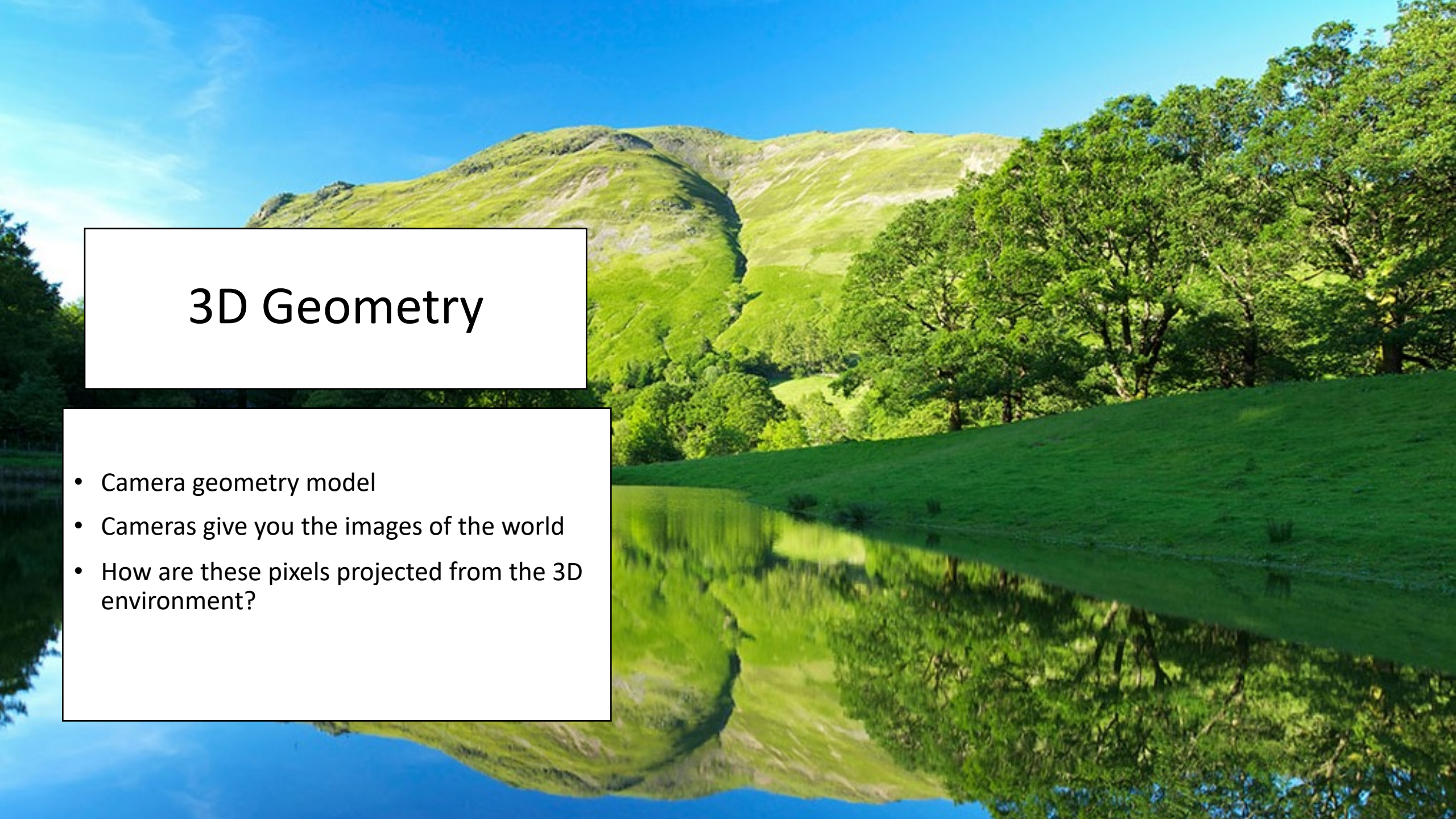
- From quaternions to angle-axis:

$$\boldsymbol{q} = \left[\cos\frac{\theta}{2}, n_x \sin\frac{\theta}{2}, n_y \sin\frac{\theta}{2}, n_z \sin\frac{\theta}{2}\right]^T.$$

- Inverse:

$$\begin{cases} \theta = 2\arccos q_0 \\ [n_x, n_y, n_z]^T = [q_1, q_2, q_3]^T / \sin\frac{\theta}{2} \end{cases}.$$

- Rotate a vector by quaternions:
  - Vector p is rotated by q and become p' , how to write their relationships?
  - Write p as quaternion (pure imaginary):
  - Then: $\boldsymbol{p}' = \boldsymbol{q}\boldsymbol{p}\boldsymbol{q}^{-1}$. Also pure imaginary



3D Geometry

- Camera geometry model
- Cameras give you the images of the world
- How are these pixels projected from the 3D environment?

3D Geometry

- Pin-hole cameras

By similar triangles:

$$\frac{Z}{f} = -\frac{X}{X'} = -\frac{Y}{Y'}.$$

Flip to the front:

$$\frac{Z}{f} = \frac{X}{X'} = \frac{Y}{Y'}.$$

Rearrange it:

$$X' = f\frac{X}{Z}$$
$$Y' = f\frac{Y}{Z}$$

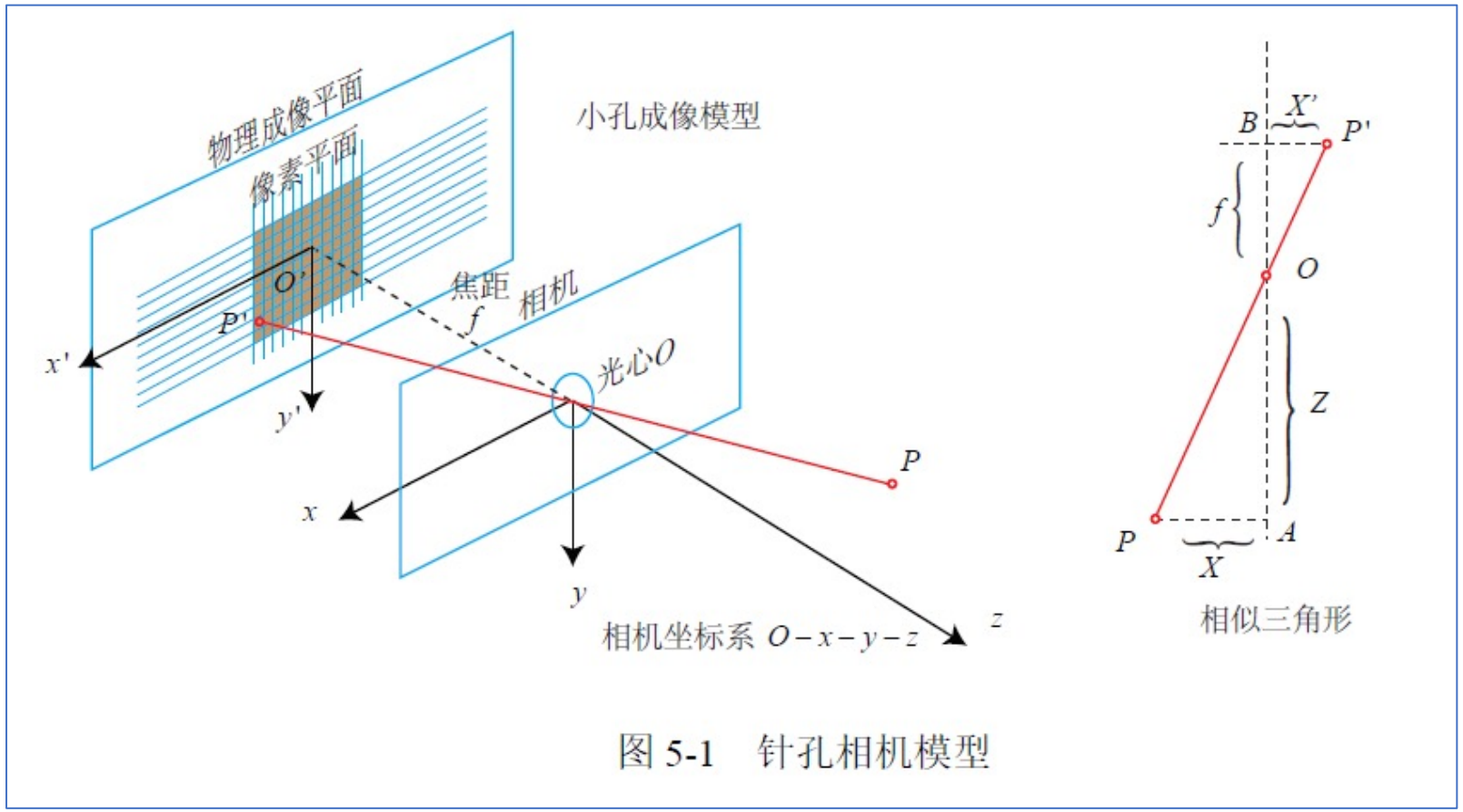


图 5-1　针孔相机模型

3D Geometry

From image plane to pixels:

$$\begin{cases} u = \alpha X' + c_x \\ v = \beta Y' + c_y \end{cases}.$$

Take into:

$$X' = f\frac{X}{Z}$$
$$Y' = f\frac{Y}{Z}.$$

Then we get:

$$\begin{cases} u = f_x\frac{X}{Z} + c_x \\ v = f_y\frac{Y}{Z} + c_y \end{cases}.$$

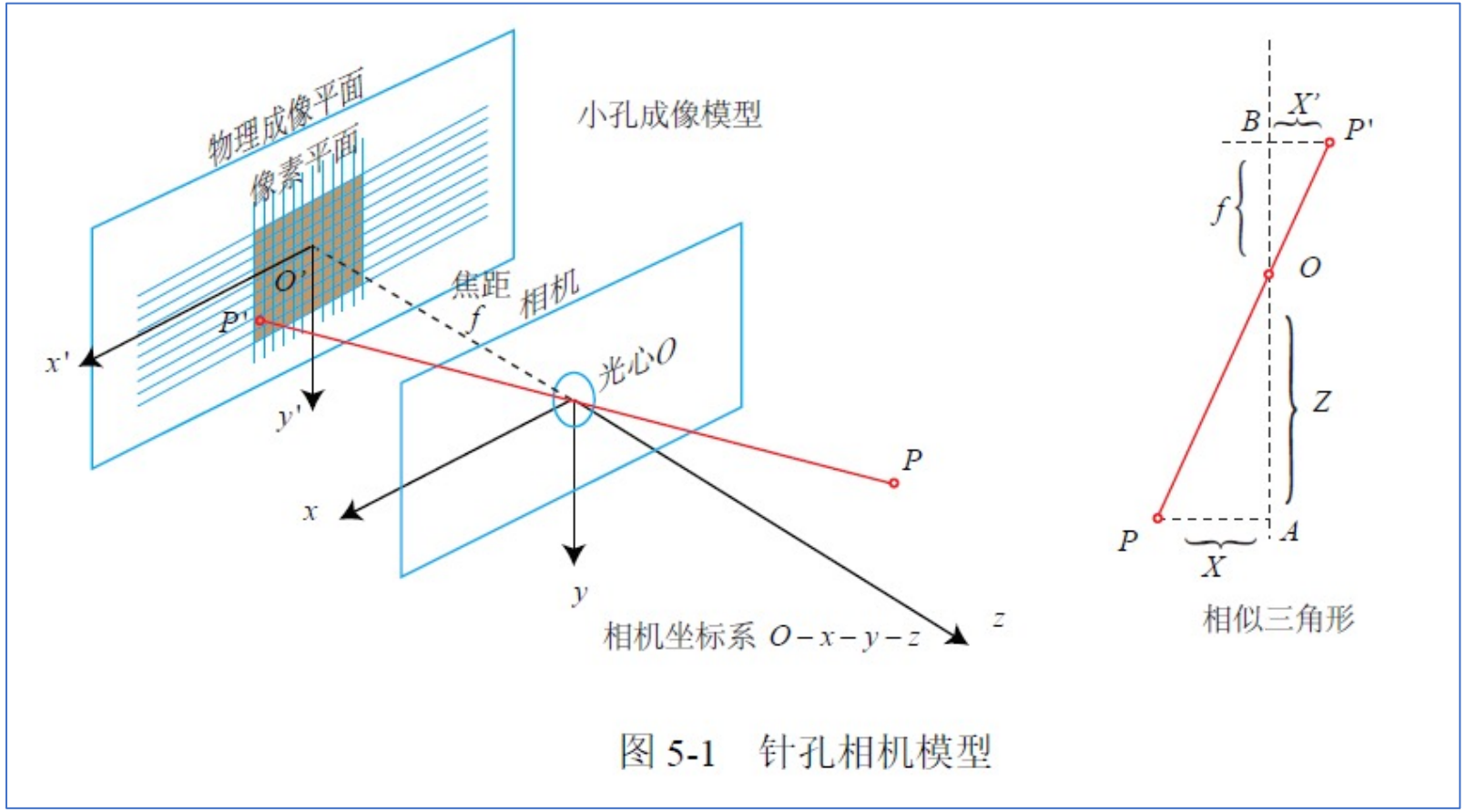


图 5-1　针孔相机模型

3D Geometry

- Pin-hole models:

$$\begin{cases} u = f_x \frac{X}{Z} + c_x \\ v = f_y \frac{Y}{Z} + c_y \end{cases}.$$

- Matrix form:

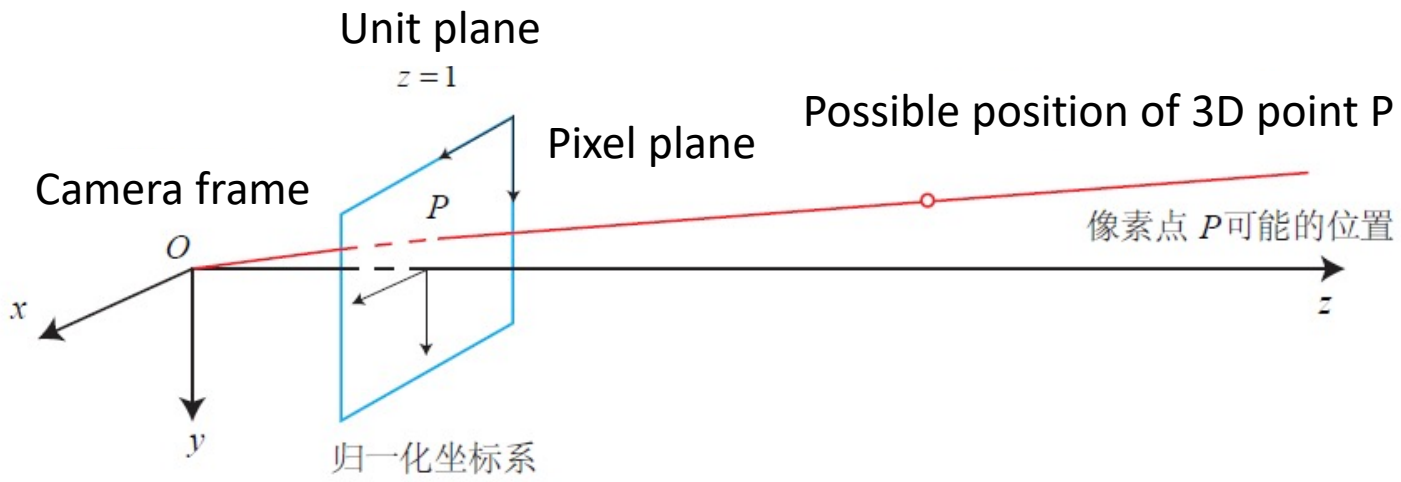
$$\begin{pmatrix} u \\ v \\ 1 \end{pmatrix} = \frac{1}{Z} \begin{pmatrix} f_x & 0 & c_x \\ 0 & f_y & c_y \\ 0 & 0 & 1 \end{pmatrix} \begin{pmatrix} X \\ Y \\ Z \end{pmatrix} \triangleq \frac{1}{Z} \boldsymbol{KP}. \qquad Z \begin{pmatrix} u \\ v \\ 1 \end{pmatrix} = \begin{pmatrix} f_x & 0 & c_x \\ 0 & f_y & c_y \\ 0 & 0 & 1 \end{pmatrix} \begin{pmatrix} X \\ Y \\ Z \end{pmatrix} \triangleq \boldsymbol{KP}.$$

- K is called as intrinsic camera matrix
  - Which is fixed for each real camera
  - And can be calibrated before running slam.

3D Geometry

- Distance is lost during the projection

$$Z\begin{pmatrix} u \\ v \\ 1 \end{pmatrix} = \begin{pmatrix} f_x & 0 & c_x \\ 0 & f_y & c_y \\ 0 & 0 & 1 \end{pmatrix}\begin{pmatrix} X \\ Y \\ Z \end{pmatrix} \triangleq \boldsymbol{KP}.$$



3D Geometry

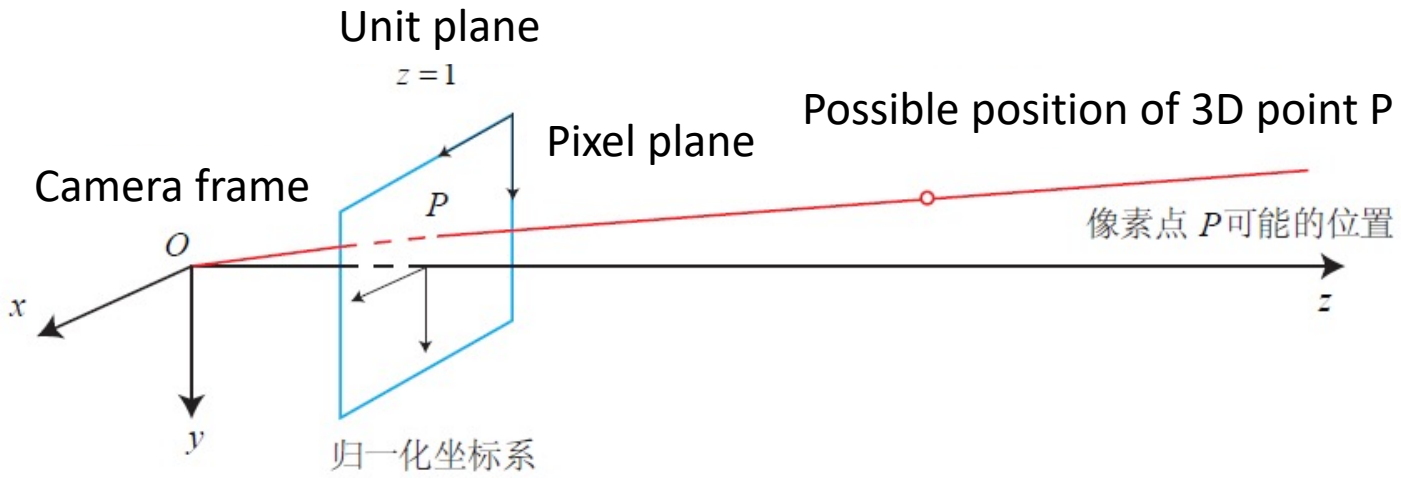
- There's another rotation and translation from the world to the camera

$$ZP_{uv} = Z\begin{bmatrix} u \\ v \\ 1 \end{bmatrix} = K\left(RP_w + t\right) = KTP_w.$$

- Here R,t or T is called as extrinsic
  - Note we assume the homogeneous coordinates are cast to non-homogenous coordinates automatically
  - In SLAM, the extrinsic R,t is our estimate purpose

3D Geometry

- Summary
  - Projection orders: world->camera->unit plane->pixels



3D Geometry

- Distortion
  - Lens will cause distortion when you have a wide range lens



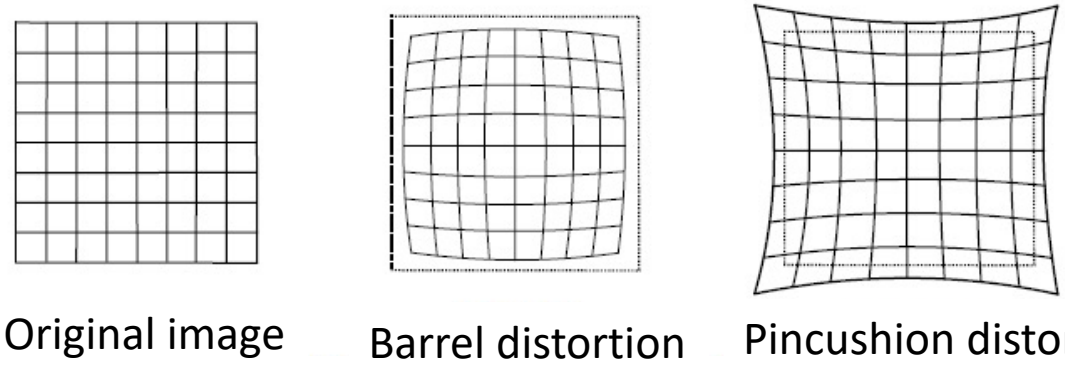
Wide range lens



Fisheye cameras

3D Geometry

- Distortion types: radial distortion and tangential distortion



Original image　　Barrel distortion　　Pincushion distortion

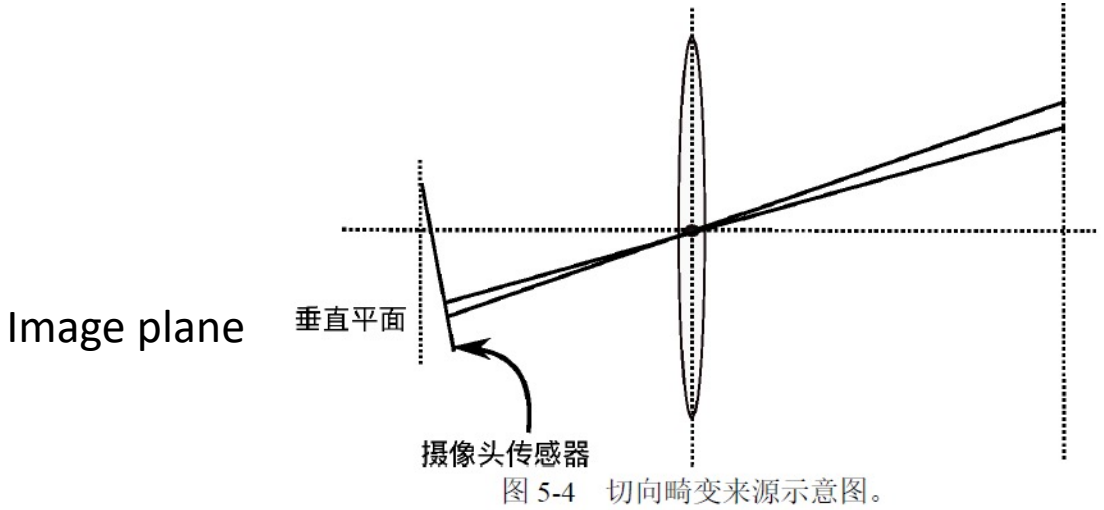


图 5-4　切向畸变来源示意图。

3D Geometry

- Mathematic form

$$x_{\mathrm{dis}torted} = x(1 + k_1 r^2 + k_2 r^4 + k_3 r^6)$$
$$y_{distorted} = y(1 + k_1 r^2 + k_2 r^4 + k_3 r^6)$$

Radial distortion

$$x_{distorted} = x + 2p_1 xy + p_2(r^2 + 2x^2)$$
$$y_{distorted} = y + p_1(r^2 + 2y^2) + 2p_2 xy$$

tangential distortion

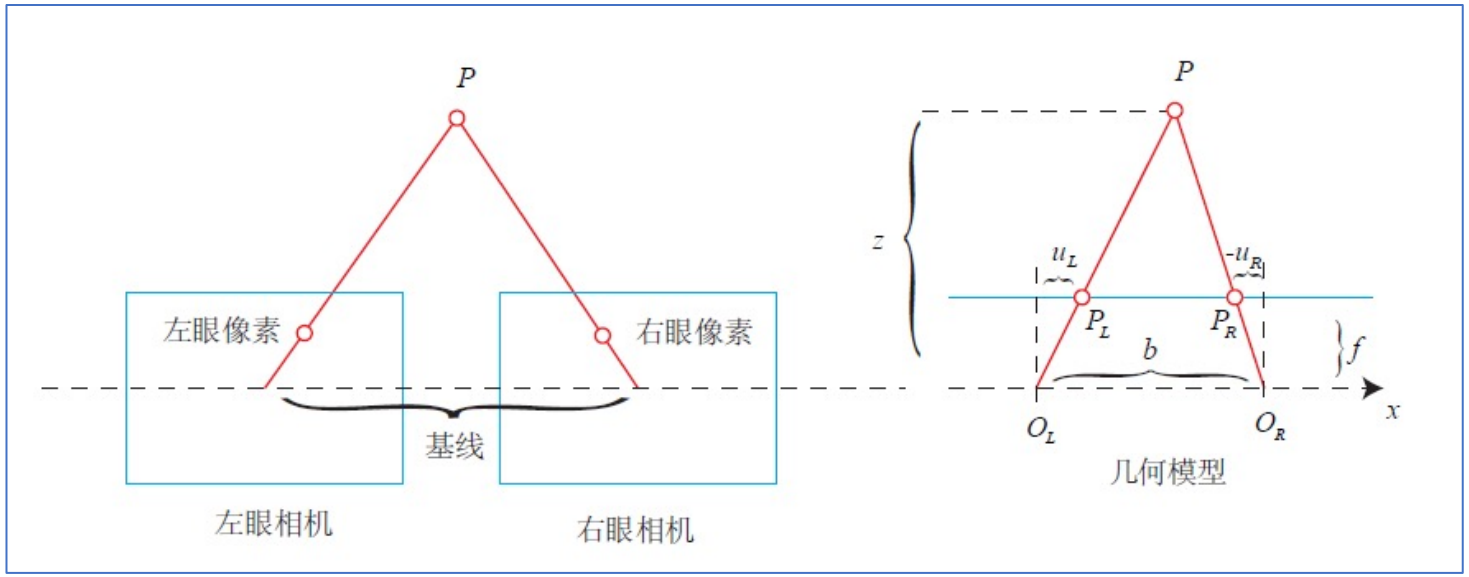
- Put them together

$$x_{distorted} = x(1 + k_1 r^2 + k_2 r^4 + k_3 r^6) + 2p_1 xy + p_2(r^2 + 2x^2)$$
$$y_{distorted} = y(1 + k_1 r^2 + k_2 r^4 + k_3 r^6) + p_1(r^2 + 2y^2) + 2p_2 xy$$

- In practice, you can choose the order of distortion params

3D Geometry

- Stereo camera
  - Two cameras (usually) placed horizontally
  - The distance between left camera center to the right is called as baseline
  - From geometric model:



$$\frac{z-f}{z}=\frac{b-u_L+u_R}{b}. \qquad z=\frac{fb}{d}, \quad d=u_L-u_R.$$

3D Geometry

RGB-D Cameras

- Structured light and ToF cameras

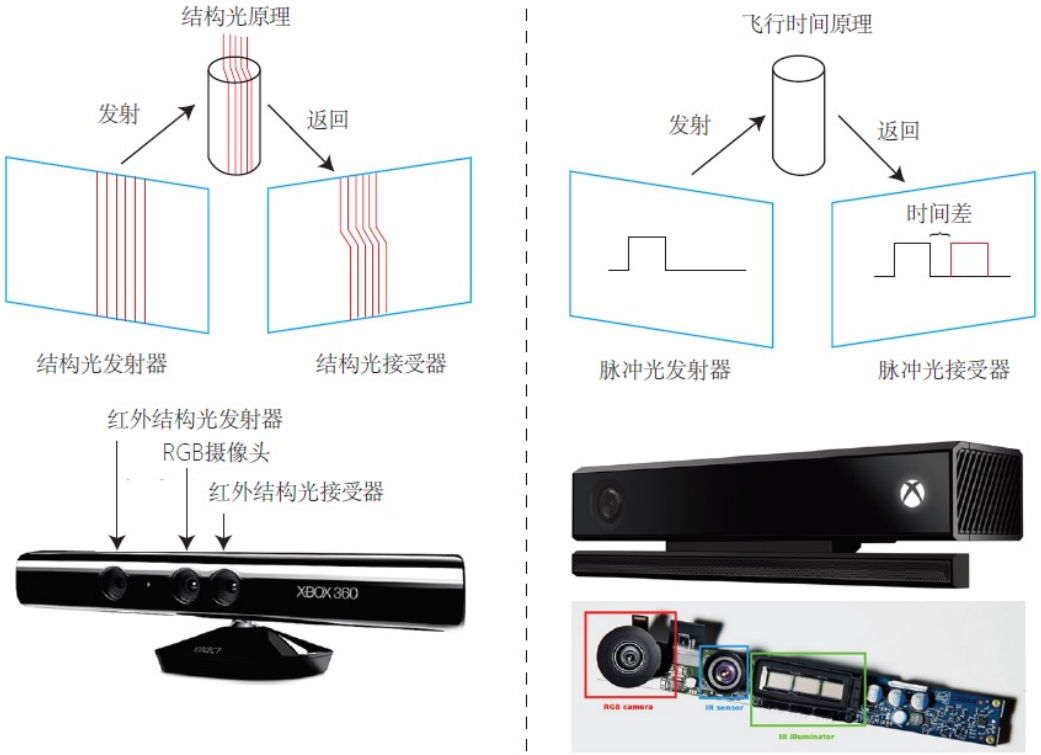
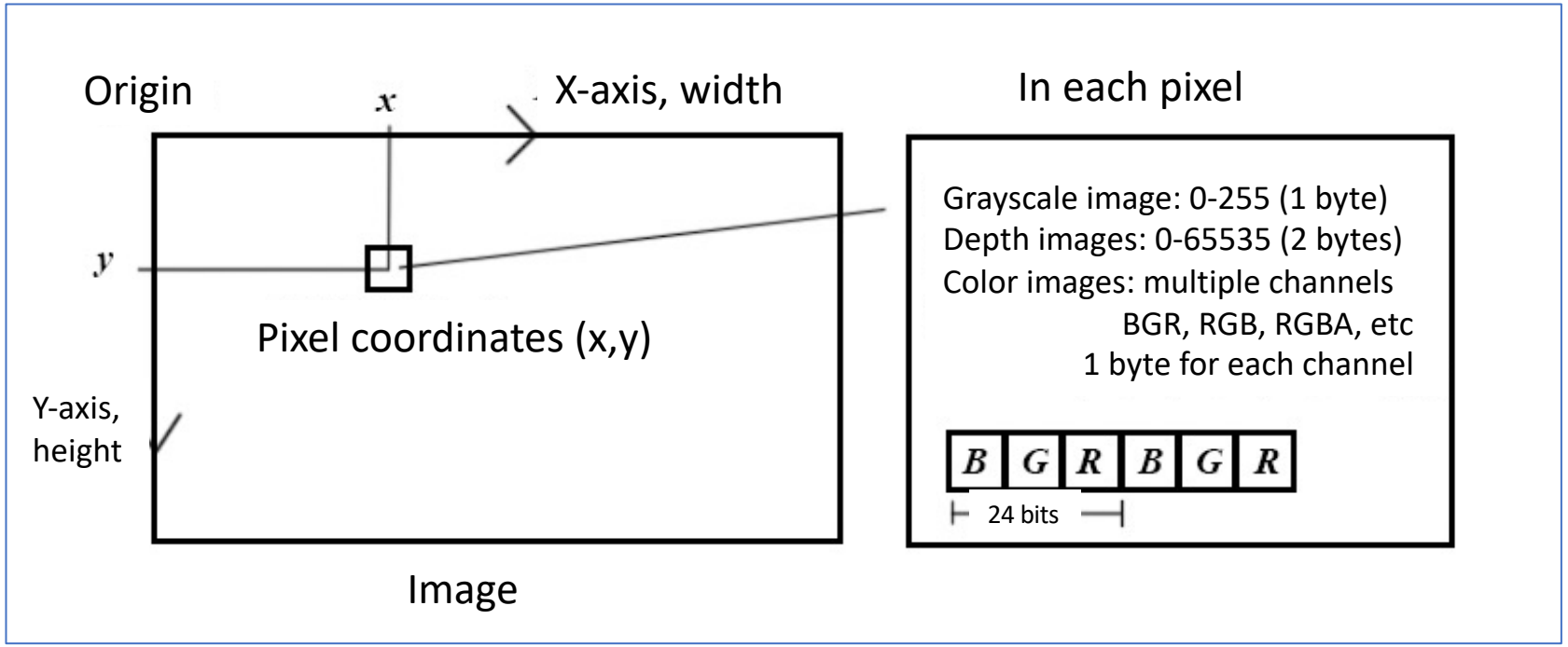


图 5-7　RGB-D 相机原理示意图

3D Geometry

- Images
- 2D arrays stored in computer
- Usually 0-255 (1 byte) grayscale values after quantification



Contents

- From state estimation to least square
- Batch least square
- Optimization on Manifolds: Lie Group and Lie Algebra

课程视频： https://www.shenlanxueyuan.com/course/386

State Estimation, Nonlinear Optimization

State Estimation

- Recall the motion model and observation model

$$\begin{cases} \boldsymbol{x}_k = f\left(\boldsymbol{x}_{k-1}, \boldsymbol{u}_k, \boldsymbol{w}_k\right) \\ \boldsymbol{z}_{k,j} = h\left(\boldsymbol{y}_j, \boldsymbol{x}_k, \boldsymbol{v}_{k,j}\right) \end{cases}.$$

- How to estimate the unknown variables given the observation data?

State Estimation

- Batch approach
  - Give all the measurements
  - To estimate all the state variables
- State variables:

$$x = \{x_1, \ldots, x_N, y_1, \ldots, y_M\}.$$

Observation and input:

$$u = \{u_1, u_2, \cdots\}, z = \{z_{k,j}\}$$

- Our purpose:

$$P(x|z, u).$$

- Bayes' Rule:

Likehood Priori

$$p(x|u, z) = \frac{P(z|x, u)p(x|u)}{P(z|u)}$$

State Estimation

- It is usually hard to write out the full distribution of Bayes' formula, but we can:
- MAP: Maximum A Posteriori

$$x_{MAP} = \operatorname{argmax}_x P(x|u,z) = \operatorname{argmax} \frac{P(z|x,u)P(x|u)}{P(z|u)}$$
$$= \operatorname{argmax} P(z|x)P(x|u)$$

Drop denominator because it is not relevant with x

Drop u because z is not relevant with u

- "In which state it is most likely to produce such measurements"

State Estimation

- From MAP to batch least square
- We assume the noise variables are independent, so that the joint pdf can be factorized:

$$P(z|x) = \prod_{k=0}^{K} P(z_k|x_k)$$

- Let's consider a single observation:
  - Affected by white Gaussian noise:

$$\boldsymbol{z}_{k,j} = h\left(\boldsymbol{y}_j, \boldsymbol{x}_k\right) + \boldsymbol{v}_{k,j},$$

$$v_{k,j} \sim N\left(0, Q_{k,j}\right)$$

- The observation model gives us a conditional pdf:

$$P(\boldsymbol{z}_{j,k}|\boldsymbol{x}_k, \boldsymbol{y}_j) = N\left(h(\boldsymbol{y}_j, \boldsymbol{x}_k), \boldsymbol{Q}_{k,j}\right).$$

- Then how to compute the MAP of x,y given z?

State Estimation

- Gaussian distribution (matrix form)

$$P(\boldsymbol{x}) = \frac{1}{\sqrt{(2\pi)^N \det(\boldsymbol{\Sigma})}} \exp\left(-\frac{1}{2}(\boldsymbol{x}-\boldsymbol{\mu})^T \boldsymbol{\Sigma}^{-1}(\boldsymbol{x}-\boldsymbol{\mu})\right).$$

- Take minus logarithm at both sides:

$$-\ln(P(\boldsymbol{x})) = \frac{1}{2}\ln\left((2\pi)^N \det(\boldsymbol{\Sigma})\right) + \frac{1}{2}(\boldsymbol{x}-\boldsymbol{\mu})^T \boldsymbol{\Sigma}^{-1}(\boldsymbol{x}-\boldsymbol{\mu}).$$

Constant w.r.t x Mahalanobis distance (sigma-norm)

- Maximum of P(x) is equivalent to minimum of –ln(P(x))

State Estimation

- Take this into the MAP:

Max: $P(\boldsymbol{z}_{j,k}|\boldsymbol{x}_k, \boldsymbol{y}_j) = N\left(h(\boldsymbol{y}_j, \boldsymbol{x}_k), \boldsymbol{Q}_{k,j}\right).$

Information matrix

$$\rightarrow \quad x_k, y_j = \operatorname{argmin}\left(\left(z_{k,j} - h(y_j, x_k)\right)^T Q_{j,k}^{-1}\left(z_{k,j} - h(y_j, x_k)\right)\right)$$

Error or residual of single observation

- The MAP problem turns out to be a least square problem

State Estimation

- Batch least square
- Original problem

$$\begin{cases} \boldsymbol{x}_k = f\left(\boldsymbol{x}_{k-1}, \boldsymbol{u}_k, \boldsymbol{w}_k\right) \\ \boldsymbol{z}_{k,j} = h\left(\boldsymbol{y}_j, \boldsymbol{x}_k, \boldsymbol{v}_{k,j}\right) \end{cases}.$$

Least square
Define the errors(residuals)

$$\begin{aligned} \boldsymbol{e}_{v,k} &= \boldsymbol{x}_k - f\left(\boldsymbol{x}_{k-1}, \boldsymbol{u}_k\right) \\ \boldsymbol{e}_{y,j,k} &= \boldsymbol{z}_{k,j} - h\left(\boldsymbol{x}_k, \boldsymbol{y}_j\right), \end{aligned}$$

$$x_{MAP} = \text{argmax} P(z|x)P(x|u)$$

- Sum of the squared residuals:

min $$J(\boldsymbol{x}) = \sum_k \boldsymbol{e}_{v,k}^T \boldsymbol{R}_k^{-1} \boldsymbol{e}_{v,k} + \sum_k \sum_j \boldsymbol{e}_{y,k,j}^T \boldsymbol{Q}_{k,j}^{-1} \boldsymbol{e}_{y,k,j}.$$

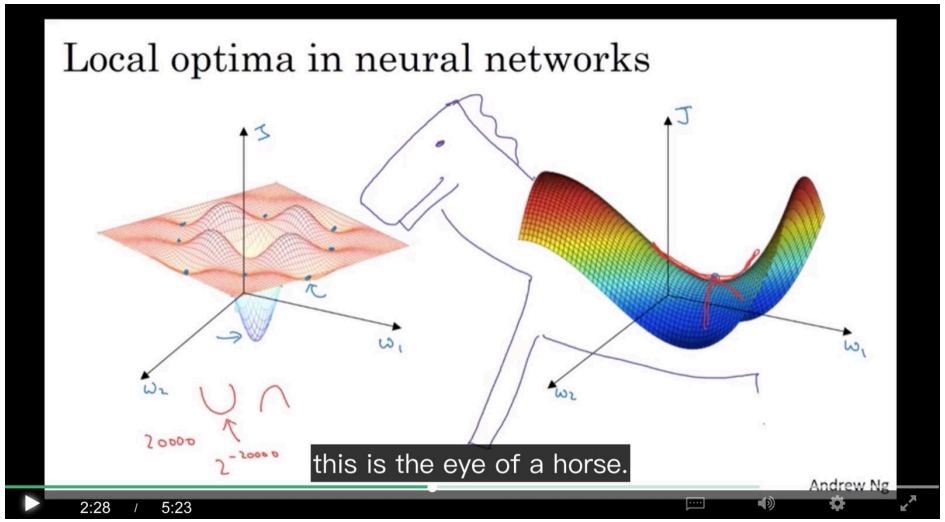
State Estimation

$$J(\boldsymbol{x}) = \sum_k \boldsymbol{e}_{v,k}^T \boldsymbol{R}_k^{-1} \boldsymbol{e}_{v,k} + \sum_k \sum_j \boldsymbol{e}_{y,k,j}^T \boldsymbol{Q}_{k,j}^{-1} \boldsymbol{e}_{y,k,j}.$$

- Some notes:
  - Because of noise, when we take the estimated trajectory and map into the models, they won't fit perfectly
  - Then we adjust our estimation to get a better estimation (minimize the error)
  - The error distribution is affected by noise distribution (information matrix)
- Structure of the least square problem
  - Sum of many squared errors
  - The dimension of total state variable maybe high
  - But single error item is easy (only related to two states in our case)
  - If we use Lie group and Lie algebra, then it's a non-constrained least square

Batch Least Square

- How to solve a least square problem?
  - Non-linear, discrete time, non-constrained
- Let's start from a simple example



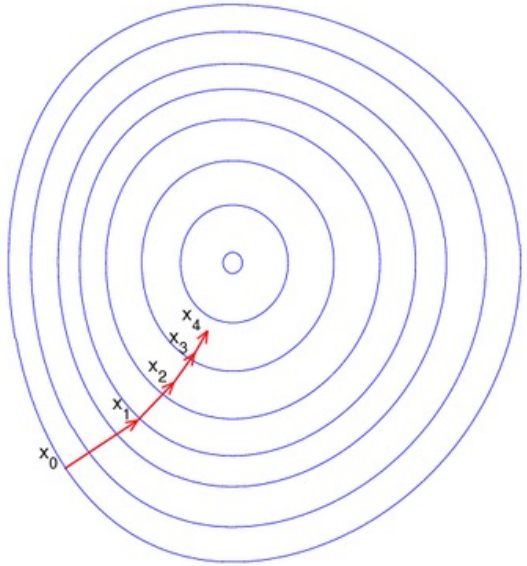
- Consider minimizing a squared error:

$$\min J(x) = \min \frac{1}{2} \|f(x)\|_2^2$$

$$x \in \mathbb{R}^n$$

- When J is simple, just solve: $\frac{dJ}{dx} = 0$
- And we will find the maxima/minima/saddle points

Batch Least Square



- When J is a complicated function:
  - dJ/dx=0 is hard to solve
  - We use iterative methods
- Iterative methods
  1. Start from a initial estimation x_0
  2. At iteration k , we find a incremental Δx_k to make $\|f(x_k + \Delta x_k)\|_2^2$ become smaller
  3. If Δx_k is small enough, stop (converged)
  4. If not, set $x_{k+1} = x_k + \Delta x_k$ and return to step 2.

Batch Least Square

- How to find the incremental part?
- By the gradient
- Taylor expansion of the object function:

$$\|f(\boldsymbol{x}+\Delta\boldsymbol{x})\|_2^2 \approx \|f(\boldsymbol{x})\|_2^2 + \boldsymbol{J}(\boldsymbol{x})\Delta\boldsymbol{x} + \frac{1}{2}\Delta\boldsymbol{x}^T\boldsymbol{H}\Delta\boldsymbol{x}.$$

Jacobian Hessian

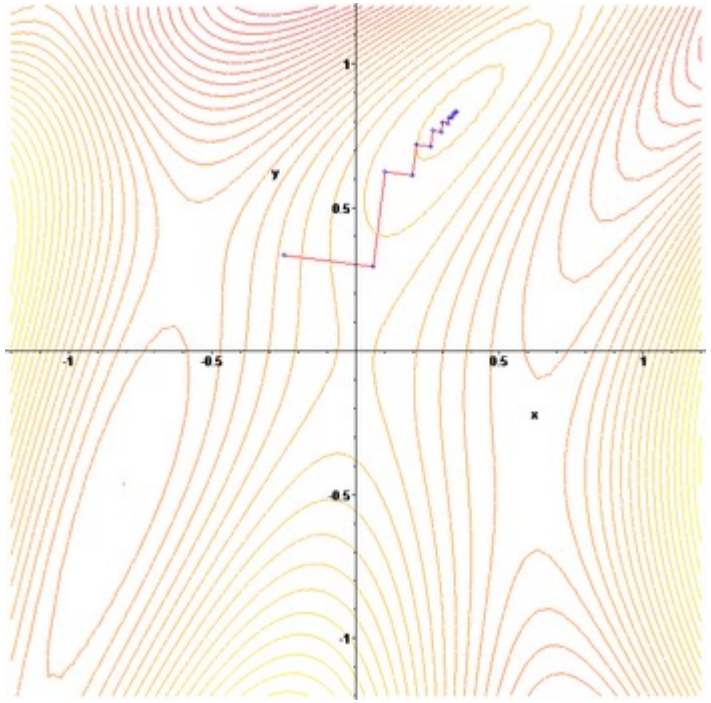
- First order methods and second order methods
- First order: (Steepest descent)

$\min_{\Delta x}\|f(x)\|_2^2 + J\Delta x$ Incremental will be: $\Delta\boldsymbol{x}^* = -\boldsymbol{J}^T(\boldsymbol{x}).$

Usually we need a step size

Batch Least Square

- Zig-zag in steepest descent



Other shortcomings

- Slow convergence speed
- Slow when close to the minimum

Batch Least Square

- Second order methods

$$\|f(\boldsymbol{x}+\Delta\boldsymbol{x})\|_2^2 \approx \|f(\boldsymbol{x})\|_2^2 + \boldsymbol{J}(\boldsymbol{x})\Delta\boldsymbol{x} + \frac{1}{2}\Delta\boldsymbol{x}^T\boldsymbol{H}\Delta\boldsymbol{x}.$$

- Solve an increment to minimize it:

$$\Delta\boldsymbol{x}^* = \arg\min \|f(\boldsymbol{x})\|_2^2 + \boldsymbol{J}(\boldsymbol{x})\Delta\boldsymbol{x} + \frac{1}{2}\Delta\boldsymbol{x}^T\boldsymbol{H}\Delta\boldsymbol{x}.$$

- Let the derivative to Δx be zero, then we get: $\boldsymbol{H}\Delta\boldsymbol{x} = -\boldsymbol{J}^T.$
- This is called Newton's method

Batch Least Square

- Second order method converges more quickly than first order methods
- But the Hessian matrix maybe hard to compute: $H\Delta x = -J^T.$

- Can we avoid the Hessian matrix and also keeps second order's convergence speed?
  - Gauss-Newton
  - Levenberg-Marquardt

Batch Least Square

- Gauss-Newton
  - Taylor expansion of f(x): $f(x+\Delta x) \approx f(x) + J(x)\Delta x.$
  - Then the squared error becomes:

$$\frac{1}{2}\|f(x) + J(x)\Delta x\|^2 = \frac{1}{2}(f(x) + J(x)\Delta x)^T (f(x) + J(x)\Delta x)$$
$$= \frac{1}{2}\left(\|f(x)\|_2^2 + 2f(x)^T J(x)\Delta x + \Delta x^T J(x)^T J(x)\Delta x\right).$$

  - Also let its derivative with Δx be zero:

$$2J(x)^T f(x) + 2J(x)^T J(x)\Delta x = 0.$$

$$J(x)^T J(x)\Delta x = -J(x)^T f(x).$$

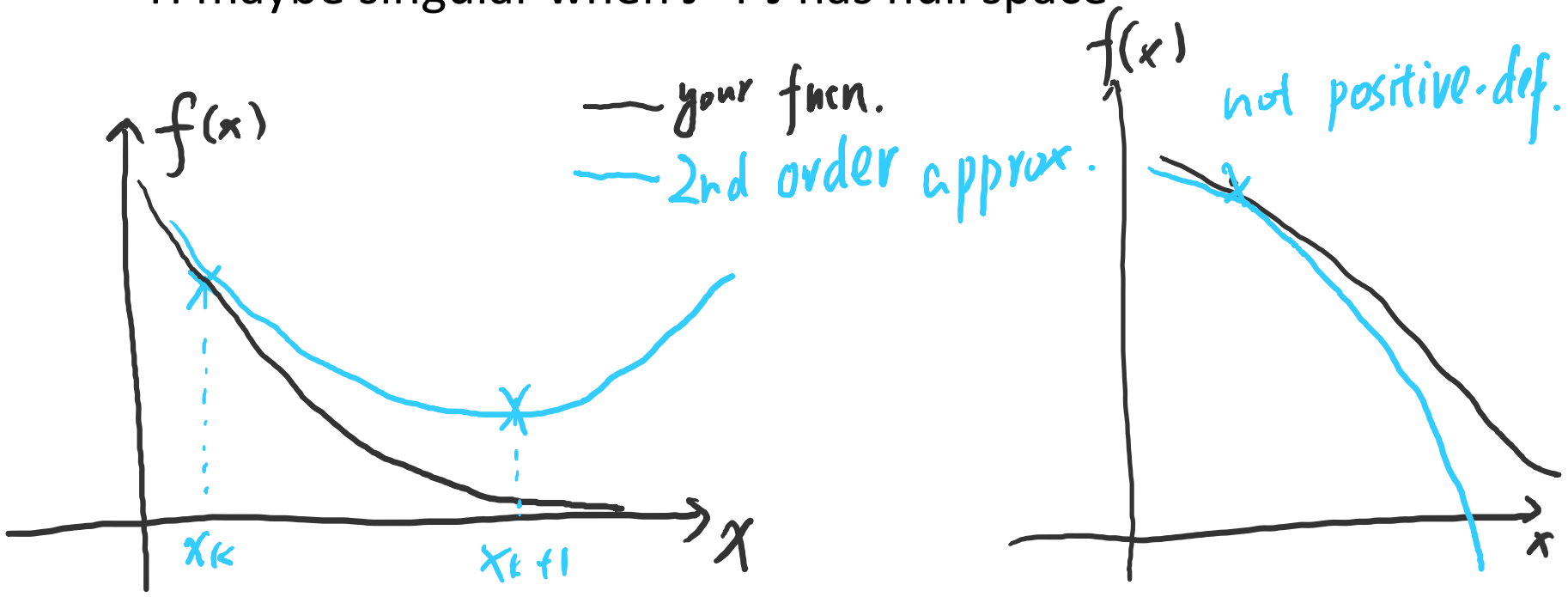
H g

$$H\Delta x = g.$$

Batch Least Square

$$\boldsymbol{J}(\boldsymbol{x})^T \boldsymbol{J}(\boldsymbol{x}) \Delta \boldsymbol{x} = -\boldsymbol{J}(\boldsymbol{x})^T f(\boldsymbol{x}).$$

- Gauss-Newton use $J(x)^T J(x)$ as an approximation of the Hessian
  - Therefore avoiding the computation of H in the Newton's method
- But $J(x)^T J(x)$ is only semi-positive definite
  - H maybe singular when J^T J has null space



Batch Least Square

- Levernberg-Marquardt method
  - Trust region approach: approximation is only valid in a region
  - Evaluate if the approximation is good:

$$\rho = \frac{f(\boldsymbol{x} + \Delta\boldsymbol{x}) - f(\boldsymbol{x})}{\boldsymbol{J}(\boldsymbol{x})\,\Delta\boldsymbol{x}}.$$

Real descent/approx. descent

  - If rho is large, increase the region
  - If rho is small, decrease the region

- LM optimization: $\min_{\Delta x_k} \frac{1}{2}\|f(x_k) + J(x_k)\Delta x_k\|^2, s.t. \|\Delta x_k\|^2 \leq \mu$
  - Assume the approximation is only good within a ball

Batch Least Square

- Trust region problem:

$$\min_{\Delta x_k} \frac{1}{2} \| f(x_k) + J(x_k) \Delta x_k \|^2, s.t. \| \Delta x_k \|^2 \leq \mu$$

- Expand it just like in G-N's case, the incremental will be:

$$(J(x_k)^T J(x_k) + \lambda I) \Delta x_k = g \qquad \lambda(\| \Delta x_k \|^2 - \mu) = 0$$

- This λI increase the semi-positive definite property of the Hessian
  - Also balancing the first-order and second-order items

Batch Least Square

- Other methods
  - Dog-leg method
  - Conjugate gradient method
  - Quasi-Newton's method
  - Pseudo-Newton's method
  - …
- You can find more in optimization books if you are interested

- In SLAM, we use G-N or L-M to solve camera's motion, pixel's movement, optical-flow, etc.

Contents

- From state estimation to least square
- Batch least square
- Optimization on Manifolds: Lie Group and Lie Algebra

Contents

- From state estimation to least square
- Batch least square
- Optimization on Manifolds: Lie Group and Lie Algebra

*Optimization on Manifold: Lie Group and Lie Algebra

Lie Group and Lie Algebra

- We use SO(3) and SE(3) to represent the pose of camera
- Let's consider optimizing some function of rotation/transform

$$f(R) \qquad \frac{df}{df(R)} \qquad \frac{f(R+\Delta R)-f(R)}{\Delta R}$$

  - Rotation and transform matrix don't have a plus operator!

Lie Group and Lie Algebra

- Group
  - 3D rotation matrix forms the Special Orthogonal Group

$$SO(3) = \{\boldsymbol{R} \in \mathbb{R}^{3\times 3} | \boldsymbol{R}\boldsymbol{R}^T = \boldsymbol{I}, det(\boldsymbol{R}) = 1\}.$$

  - 3D transform matrix forms the Special Euclidean Group

$$SE(3) = \left\{ \boldsymbol{T} = \begin{bmatrix} \boldsymbol{R} & \boldsymbol{t} \\ \boldsymbol{0}^T & 1 \end{bmatrix} \in \mathbb{R}^{4\times 4} | \boldsymbol{R} \in SO(3), \boldsymbol{t} \in \mathbb{R}^3 \right\}.$$

- What is Group?

Lie Group and Lie Algebra

- Group
  - Group is a set with an operator $(A,\cdot)$ that satisfies the following:

    1. Closure $\forall a_1, a_2 \in A, \quad a_1 \cdot a_2 \in A.$

    2. Associativity $\forall a_1, a_2, a_3 \in A, \quad (a_1 \cdot a_2) \cdot a_3 = a_1 \cdot (a_2 \cdot a_3).$

    3. Identity $\exists a_0 \in A, \quad s.t. \quad \forall a \in A, \quad a_0 \cdot a = a \cdot a_0 = a.$

    4. Invertibility $\forall a \in A, \quad \exists a^{-1} \in A, \quad s.t. \quad a \cdot a^{-1} = a_0.$

- Obviously,
  - $(SO(3),\cdot), (SE(3),\cdot)$ are groups

Lie Group and Lie Algebra

- Lie Group
  - Group that is smooth
  - Group that is also a manifold
  - “Locally looks like R^n ”
  - Further explanation needs knowledges from topology and differential geometry
  - SO(3) and SE(3) are also Lie groups

- Lie Algebra
  - Tangent space of the Lie group at identity
  - SO(3)->so(3), SE(3)->se(3)

Lie Group and Lie Algebra

- Introducing of the Lie Algebra
  - Assume a time-varying rotation matrix $R(t)$
  - It satisfies: $\boldsymbol{R}(t)\boldsymbol{R}(t)^T = \boldsymbol{I}.$
  - Take derivative of time t at both sides:

$$\dot{\boldsymbol{R}}(t)\boldsymbol{R}(t)^T + \boldsymbol{R}(t)\dot{\boldsymbol{R}}(t)^T = 0.$$

  - Rearrange:

$$\dot{\boldsymbol{R}}(t)\boldsymbol{R}(t)^T = -\left(\dot{\boldsymbol{R}}(t)\boldsymbol{R}(t)^T\right)^T.$$

Skew-symmetric

Lie Group and Lie Algebra

$$\boldsymbol{a}^\wedge = \boldsymbol{A} = \begin{bmatrix} 0 & -a_3 & a_2 \\ a_3 & 0 & -a_1 \\ -a_2 & a_1 & 0 \end{bmatrix}, \quad \boldsymbol{A}^\vee = \boldsymbol{a}.$$

- Denote the skew-symmetric matrix as $\phi(t)^\wedge$

$$\dot{\boldsymbol{R}}(t)\boldsymbol{R}(t)^T = -\left(\dot{\boldsymbol{R}}(t)\boldsymbol{R}(t)^T\right)^T. \qquad \dot{\boldsymbol{R}}(t)\boldsymbol{R}(t)^T = \boldsymbol{\phi}(t)^\wedge.$$

- Put R(t) to the right side: $\dot{\boldsymbol{R}}(t) = \boldsymbol{\phi}(t)^\wedge \boldsymbol{R}(t)$
- It looks like when we take the derivative, we will get a $\phi(t)^\wedge$ at the left side

- Assume we are close to identity: $t_0 = 0, R(0) = I$
- And $\phi(t)^\wedge$ does not change: $\dot{\boldsymbol{R}}(t) = \boldsymbol{\phi}(t_0)^\wedge \boldsymbol{R}(t) = \boldsymbol{\phi}_0^\wedge \boldsymbol{R}(t).$

- With $R(0) = I$, we solve this ODE:

$$\boldsymbol{R}(t) = \exp\left(\boldsymbol{\phi}_0^\wedge t\right).$$

Lie Group and Lie Algebra

$$\boldsymbol{R}(t) = \exp\left(\boldsymbol{\phi}_0^{\wedge} t\right).$$

- So, if t is close to 0, then we can always find an R given an ϕ

- ϕ is called a Lie algebra
- From a Lie algebra, if we take a Exponential Map, then it becomes a Lie group

- Questions:
  - Lie algebra's definition and constraints?
  - How to compute the exponential map?

Lie Group and Lie Algebra

- Lie algebra:
  - We have a Lie algebra for each Lie group, which is a vector space (the tangent space) at the identity $\forall \boldsymbol{X}, \boldsymbol{Y} \in \mathbb{V}, [\boldsymbol{X}, \boldsymbol{Y}] \in \mathbb{V}.$
  - Lie algebra has a vector space over field together with a binary operator (Lie bracket) $[,]$, that satisfies:
  - Closure: $\forall \boldsymbol{X}, \boldsymbol{Y} \in \mathbb{V}, [\boldsymbol{X}, \boldsymbol{Y}] \in \mathbb{V}.$
  - Bilinearity: for any $\forall \boldsymbol{X}, \boldsymbol{Y}, \boldsymbol{Z} \in \mathbb{V}, a, b \in \mathbb{F},$

$$[a\boldsymbol{X} + b\boldsymbol{Y}, \boldsymbol{Z}] = a[\boldsymbol{X}, \boldsymbol{Z}] + b[\boldsymbol{Y}, \boldsymbol{Z}], \quad [\boldsymbol{Z}, a\boldsymbol{X} + b\boldsymbol{Y}] = a[\boldsymbol{Z}, \boldsymbol{X}] + b[\boldsymbol{Z}, \boldsymbol{Y}].$$

  - Alternativity: $\forall \boldsymbol{X} \in \mathbb{V}, [\boldsymbol{X}, \boldsymbol{X}] = \boldsymbol{0}.$
  - Jacobi identity: $\forall \boldsymbol{X}, \boldsymbol{Y}, \boldsymbol{Z} \in \mathbb{V}, [\boldsymbol{X}, [\boldsymbol{Y}, \boldsymbol{Z}]] + [\boldsymbol{Z}, [\boldsymbol{Y}, \boldsymbol{X}]] + [\boldsymbol{Y}, [\boldsymbol{Z}, \boldsymbol{X}]] = \boldsymbol{0}.$

Lie Group and Lie Algebra

- Example: $(R^3, R, \times)$ is a Lie algebra
- Lie algebra so(3): $\mathfrak{so}(3) = \left\{ \phi \in \mathbb{R}^3, \boldsymbol{\Phi} = \phi^\wedge \in \mathbb{R}^{3\times 3} \right\}.$
  - where $$\boldsymbol{\Phi} = \phi^\wedge = \begin{bmatrix} 0 & -\phi_3 & \phi_2 \\ \phi_3 & 0 & -\phi_1 \\ -\phi_2 & \phi_1 & 0 \end{bmatrix} \in \mathbb{R}^{3\times 3}.$$
  - And the Lie bracket is: $[\phi_1, \phi_2] = (\boldsymbol{\Phi}_1 \boldsymbol{\Phi}_2 - \boldsymbol{\Phi}_2 \boldsymbol{\Phi}_1)^\vee.$

Lie Group and Lie Algebra

- Similarly, for SE(3) we also have se(3):

$$\mathfrak{se}(3) = \left\{ \boldsymbol{\xi} = \begin{bmatrix} \boldsymbol{\rho} \\ \boldsymbol{\phi} \end{bmatrix} \in \mathbb{R}^6, \boldsymbol{\rho} \in \mathbb{R}^3, \boldsymbol{\phi} \in \mathfrak{so}(3), \boldsymbol{\xi}^\wedge = \begin{bmatrix} \boldsymbol{\phi}^\wedge & \boldsymbol{\rho} \\ \mathbf{0}^T & 0 \end{bmatrix} \in \mathbb{R}^{4\times 4} \right\}.$$

- Where $\boldsymbol{\xi}^\wedge = \begin{bmatrix} \boldsymbol{\phi}^\wedge & \boldsymbol{\rho} \\ \mathbf{0}^T & 0 \end{bmatrix} \in \mathbb{R}^{4\times 4}.$ and Lie bracket is: $[\boldsymbol{\xi}_1, \boldsymbol{\xi}_2] = (\boldsymbol{\xi}_1^\wedge \boldsymbol{\xi}_2^\wedge - \boldsymbol{\xi}_2^\wedge \boldsymbol{\xi}_1^\wedge)^\vee.$

NOTE in se(3) this operator is not a skew-symmetric matrix, but we keep its form

- Note:
  - The definition of se(3) may be different in literature
  - Vector or matrix are both ok to define a lie algebra

Lie Group and Lie Algebra

- Exponential map
  - Operator from Lie algebra to Lie group: $R = \exp(\phi^\wedge)$
  - Here $\phi^\wedge$ is a 3x3 matrix so this exponential map is a matrix operator
  - Take Taylor expansion:

$$\exp(\phi^\wedge) = \sum_{n=0}^{\infty} \frac{1}{n!}(\phi^\wedge)^n.$$

  - Directly computing this Taylor expansion is intractable

Lie Group and Lie Algebra

- Take the length and direction of ϕ , then $\phi = \theta a$
- For a unit-length vector, we have:

$$a^{\wedge} a^{\wedge} = a a^T - I,$$

$$a^{\wedge} a^{\wedge} a^{\wedge} = -a^{\wedge}.$$

This will be useful when handling the high-order Taylor expansion items

Lie Group and Lie Algebra

- Compute the Taylor expansion:

$$
\begin{aligned}
\exp(\phi^\wedge) = \exp(\theta \boldsymbol{a}^\wedge) &= \sum_{n=0}^{\infty} \frac{1}{n!} (\theta \boldsymbol{a}^\wedge)^n \\
&= \boldsymbol{I} + \theta \boldsymbol{a}^\wedge + \frac{1}{2!}\theta^2 \boldsymbol{a}^\wedge \boldsymbol{a}^\wedge + \frac{1}{3!}\theta^3 \boldsymbol{a}^\wedge \boldsymbol{a}^\wedge \boldsymbol{a}^\wedge + \frac{1}{4!}\theta^4 (\boldsymbol{a}^\wedge)^4 + \ldots \\
&= \boldsymbol{a}\boldsymbol{a}^T - \boldsymbol{a}^\wedge \boldsymbol{a}^\wedge + \theta \boldsymbol{a}^\wedge + \frac{1}{2!}\theta^2 \boldsymbol{a}^\wedge \boldsymbol{a}^\wedge - \frac{1}{3!}\theta^3 \boldsymbol{a}^\wedge - \frac{1}{4!}\theta^4 (\boldsymbol{a}^\wedge)^2 + \ldots \\
&= \boldsymbol{a}\boldsymbol{a}^T + \left(\theta - \frac{1}{3!}\theta^3 + \frac{1}{5!}\theta^5 - \ldots\right) \boldsymbol{a}^\wedge - \left(1 - \frac{1}{2!}\theta^2 + \frac{1}{4!}\theta^4 - \ldots\right) \boldsymbol{a}^\wedge \boldsymbol{a}^\wedge \\
&= \boldsymbol{a}^\wedge \boldsymbol{a}^\wedge + \boldsymbol{I} + \sin\theta \boldsymbol{a}^\wedge - \cos\theta \boldsymbol{a}^\wedge \boldsymbol{a}^\wedge \\
&= (1 - \cos\theta) \boldsymbol{a}^\wedge \boldsymbol{a}^\wedge + \boldsymbol{I} + \sin\theta \boldsymbol{a}^\wedge \\
&= \cos\theta \boldsymbol{I} + (1 - \cos\theta) \boldsymbol{a}\boldsymbol{a}^T + \sin\theta \boldsymbol{a}^\wedge .
\end{aligned}
$$

- Finally we get:

$$\exp(\theta \boldsymbol{a}^\wedge) = \cos\theta \boldsymbol{I} + (1 - \cos\theta) \boldsymbol{a}\boldsymbol{a}^T + \sin\theta \boldsymbol{a}^\wedge .$$

- Which is exactly the Rodrigues' formula!

Lie Group and Lie Algebra

- So so(3) is just the rotation vector
- Same as exponential map, we can also define logarithm map as:

$$\boldsymbol{\phi} = \ln(\boldsymbol{R})^{\vee} = \left(\sum_{n=0}^{\infty} \frac{(-1)^n}{n+1} (\boldsymbol{R} - \boldsymbol{I})^{n+1} \right)^{\vee}.$$

- And also don't need to actually compute this stuff, we take the conversion equations from rotation matrix to rotation vector:

$$\theta = \arccos\left(\frac{\mathrm{tr}(\boldsymbol{R}) - 1}{2}\right). \qquad \boldsymbol{R}\boldsymbol{n} = \boldsymbol{n}.$$

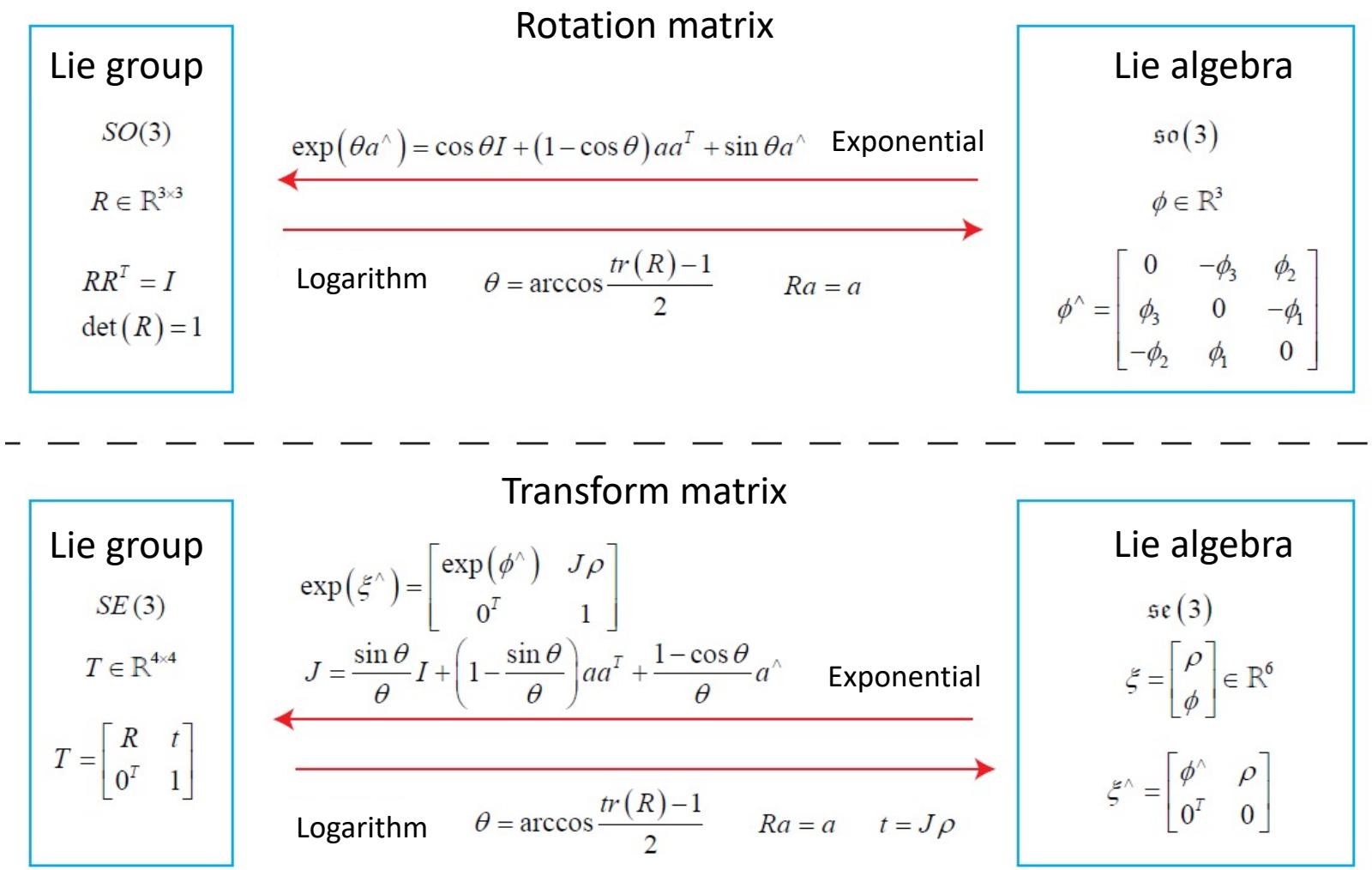
Lie Group and Lie Algebra

- For SE(3), the exponential map is:

$$\exp(\boldsymbol{\xi}^\wedge) = \begin{bmatrix} \sum_{n=0}^{\infty} \frac{1}{n!}(\boldsymbol{\phi}^\wedge)^n & \sum_{n=0}^{\infty} \frac{1}{(n+1)!}(\boldsymbol{\phi}^\wedge)^n \boldsymbol{\rho} \\ \mathbf{0}^T & 1 \end{bmatrix} \triangleq \begin{bmatrix} \boldsymbol{R} & \boldsymbol{J}\boldsymbol{\rho} \\ \mathbf{0}^T & 1 \end{bmatrix} = \boldsymbol{T}.$$

- The rotation part is just a SO(3), but the translation part has a Jacobian matrix: (left as an assignment)

$$\boldsymbol{J} = \frac{\sin\theta}{\theta}\boldsymbol{I} + \left(1 - \frac{\sin\theta}{\theta}\right)\boldsymbol{a}\boldsymbol{a}^T + \frac{1-\cos\theta}{\theta}\boldsymbol{a}^\wedge.$$



1. Lie Group and Lie Algebra

Cheat sheet of Lie group and Lie algebra

Lie Group and Lie Algebra

- Next question
  - We still don’t have plus operation for Lie group
  - Then we can’t define derivatives

- Solution
  - Take advantage of the plus in the Lie algebra, and convert it back to Lie group

- A primal question:
  - Plus in Lie algebra is equal to multiplication in Lie group?

$$\exp\left(\boldsymbol{\phi}_1^{\wedge}\right)\exp\left(\boldsymbol{\phi}_2^{\wedge}\right)=\exp\left(\left(\boldsymbol{\phi}_1+\boldsymbol{\phi}_2\right)^{\wedge}\right). \quad ?$$

Lie Group and Lie Algebra

- Unfortunately, this does not work for matrices
- Baker-Campbell-Hausdorff formula gives the full version of this multiplication:

$$\ln(\exp(\boldsymbol{A})\exp(\boldsymbol{B})) = \sum_{n=1}^{\infty}\frac{(-1)^{n-1}}{n}\sum_{\substack{r_i+s_i>0,\\ 1\leqslant i\leqslant n}}\frac{(\sum_{i=1}^{n}(r_i+s_i))^{-1}}{\prod_{i=1}^{n}r_i!s_i!}[\boldsymbol{A}^{r_1}\boldsymbol{B}^{s_1}\boldsymbol{A}^{r_2}\boldsymbol{B}^{s_2}\cdots\boldsymbol{A}^{r_n}\boldsymbol{B}^{s_n}]$$

- where

$$[\boldsymbol{A}^{r_1}\boldsymbol{B}^{s_1}\boldsymbol{A}^{r_2}\boldsymbol{B}^{s_2}\cdots\boldsymbol{A}^{r_n}\boldsymbol{B}^{s_n}] = \underbrace{[\boldsymbol{A},\cdots[\boldsymbol{A}}_{r_1},\underbrace{[\boldsymbol{B},\cdots[\boldsymbol{B}}_{s_1},\cdots\underbrace{[\boldsymbol{A},\cdots[\boldsymbol{A}}_{r_n},\underbrace{[\boldsymbol{B},\cdots[\boldsymbol{B},\boldsymbol{B}]\cdots]}_{s_n}]\cdots]\cdots]\cdots]]\cdots]$$

Lie Group and Lie Algebra

- First part of BCH formula:

$$\ln\left(\exp\left(\boldsymbol{A}\right)\exp\left(\boldsymbol{B}\right)\right)=\boldsymbol{A}+\boldsymbol{B}+\frac{1}{2}\left[\boldsymbol{A},\boldsymbol{B}\right]+\frac{1}{12}\left[\boldsymbol{A},\left[\boldsymbol{A},\boldsymbol{B}\right]\right]-\frac{1}{12}\left[\boldsymbol{B},\left[\boldsymbol{A},\boldsymbol{B}\right]\right]+\cdots$$

- If A or B is small enough we can keep the linear item only, the BCH can be approximately written as:

$$\ln\left(\exp\left(\boldsymbol{\phi}_1^{\wedge}\right)\exp\left(\boldsymbol{\phi}_2^{\wedge}\right)\right)^{\vee}\approx\begin{cases}\boldsymbol{J}_l(\boldsymbol{\phi}_2)^{-1}\boldsymbol{\phi}_1+\boldsymbol{\phi}_2 & \text{if } \boldsymbol{\phi}_1 \text{ is small,}\\ \boldsymbol{J}_r(\boldsymbol{\phi}_1)^{-1}\boldsymbol{\phi}_2+\boldsymbol{\phi}_1 & \text{if } \boldsymbol{\phi}_2 \text{ is small.}\end{cases}$$

- where

$$\boldsymbol{J}_l=\boldsymbol{J}=\frac{\sin\theta}{\theta}\boldsymbol{I}+\left(1-\frac{\sin\theta}{\theta}\right)\boldsymbol{a}\boldsymbol{a}^T+\frac{1-\cos\theta}{\theta}\boldsymbol{a}^{\wedge}.$$ Left Jacobian

$$\boldsymbol{J}_l^{-1}=\frac{\theta}{2}\cot\frac{\theta}{2}\boldsymbol{I}+\left(1-\frac{\theta}{2}\cot\frac{\theta}{2}\right)\boldsymbol{a}\boldsymbol{a}^T-\frac{\theta}{2}\boldsymbol{a}^{\wedge}.$$ Right Jacobian

$$\boldsymbol{J}_r(\boldsymbol{\phi})=\boldsymbol{J}_l(-\boldsymbol{\phi}).$$

Lie Group and Lie Algebra

- Rewrite it (we take left multiplication as an example)

$$\exp\left(\Delta\boldsymbol{\phi}^{\wedge}\right)\exp\left(\boldsymbol{\phi}^{\wedge}\right)=\exp\left(\left(\boldsymbol{\phi}+\boldsymbol{J}_{l}^{-1}\left(\boldsymbol{\phi}\right)\Delta\boldsymbol{\phi}\right)^{\wedge}\right).$$

- Left multiplication in Lie group means an addition in Lie algebra with an Jacobian
- Inversely, if we do addition in Lie algebra, the in Lie group:

$$\exp\left(\left(\boldsymbol{\phi}+\Delta\boldsymbol{\phi}\right)^{\wedge}\right)=\exp\left(\left(\boldsymbol{J}_{l}\Delta\boldsymbol{\phi}\right)^{\wedge}\right)\exp\left(\boldsymbol{\phi}^{\wedge}\right)=\exp\left(\boldsymbol{\phi}^{\wedge}\right)\exp\left(\left(\boldsymbol{J}_{r}\Delta\boldsymbol{\phi}\right)^{\wedge}\right).$$

Lie Group and Lie Algebra

- Similar in SE(3)'s case:

$$\exp\left(\Delta\boldsymbol{\xi}^{\wedge}\right)\exp\left(\boldsymbol{\xi}^{\wedge}\right) \approx \exp\left(\left(\mathcal{J}_l^{-1}\Delta\boldsymbol{\xi}+\boldsymbol{\xi}\right)^{\wedge}\right),$$

$$\exp\left(\boldsymbol{\xi}^{\wedge}\right)\exp\left(\Delta\boldsymbol{\xi}^{\wedge}\right) \approx \exp\left(\left(\mathcal{J}_r^{-1}\Delta\boldsymbol{\xi}+\boldsymbol{\xi}\right)^{\wedge}\right).$$

- Where:

$$\mathcal{J}_r(\boldsymbol{\xi}) = \begin{bmatrix} \boldsymbol{J}_r & \boldsymbol{Q}_r \\ \mathbf{0} & \boldsymbol{J}_r \end{bmatrix}$$

$$\mathcal{J}_\ell(\boldsymbol{\xi}) = \begin{bmatrix} \boldsymbol{J}_\ell & \boldsymbol{Q}_\ell \\ \mathbf{0} & \boldsymbol{J}_\ell \end{bmatrix}$$

$$\begin{aligned}\boldsymbol{Q}_\ell(\boldsymbol{\xi}) &= \sum_{n=0}^{\infty}\sum_{m=0}^{\infty}\frac{1}{(n+m+2)!}(\boldsymbol{\phi}^{\wedge})^n\boldsymbol{\rho}^{\wedge}(\boldsymbol{\phi}^{\wedge})^m \\ &= \frac{1}{2}\boldsymbol{\rho}^{\wedge} + \left(\frac{\phi-\sin\phi}{\phi^3}\right)(\boldsymbol{\phi}^{\wedge}\boldsymbol{\rho}^{\wedge}+\boldsymbol{\rho}^{\wedge}\boldsymbol{\phi}^{\wedge}+\boldsymbol{\phi}^{\wedge}\boldsymbol{\rho}^{\wedge}\boldsymbol{\phi}^{\wedge}) \\ &\quad + \left(\frac{\phi^2+2\cos\phi-2}{2\phi^4}\right)(\boldsymbol{\phi}^{\wedge}\boldsymbol{\phi}^{\wedge}\boldsymbol{\rho}^{\wedge}+\boldsymbol{\rho}^{\wedge}\boldsymbol{\phi}^{\wedge}\boldsymbol{\phi}^{\wedge}-3\boldsymbol{\phi}^{\wedge}\boldsymbol{\rho}^{\wedge}\boldsymbol{\phi}^{\wedge}) \\ &\quad + \left(\frac{2\phi-3\sin\phi+\phi\cos\phi}{2\phi^5}\right)(\boldsymbol{\phi}^{\wedge}\boldsymbol{\rho}^{\wedge}\boldsymbol{\phi}^{\wedge}\boldsymbol{\phi}^{\wedge}+\boldsymbol{\phi}^{\wedge}\boldsymbol{\phi}^{\wedge}\boldsymbol{\rho}^{\wedge}\boldsymbol{\phi}^{\wedge})\end{aligned}$$

$$\boldsymbol{Q}_r(\boldsymbol{\xi}) = \boldsymbol{Q}_\ell(-\boldsymbol{\xi}) = \boldsymbol{C}\boldsymbol{Q}_\ell(\boldsymbol{\xi}) + (\boldsymbol{J}_\ell\boldsymbol{\rho})^{\wedge}\boldsymbol{C}\boldsymbol{J}_\ell$$

Lie Group and Lie Algebra

- With BCH formula, we can define the derivate of a function of a rotation or transform matrix
- Example: rotating a point
- We want to know the derivative: $\frac{\partial (\boldsymbol{R}\boldsymbol{p})}{\partial \boldsymbol{R}}$.

- We have two solutions:
  - Add a small item in the Lie algebra, and set its limit to zero (Derivative model)
  - (Left) Multiply a small item in the Lie group, and set its Lie algebra's limit to zero (Disturb model)

Lie Group and Lie Algebra

- Derivative model:

$$
\begin{aligned}
\frac{\partial \left(\exp\left(\boldsymbol{\phi}^{\wedge}\right)\boldsymbol{p}\right)}{\partial \boldsymbol{\phi}} &= \lim_{\delta\boldsymbol{\phi}\to 0} \frac{\exp\left(\left(\boldsymbol{\phi}+\delta\boldsymbol{\phi}\right)^{\wedge}\right)\boldsymbol{p} - \exp\left(\boldsymbol{\phi}^{\wedge}\right)\boldsymbol{p}}{\delta\boldsymbol{\phi}} \\
&= \lim_{\delta\boldsymbol{\phi}\to 0} \frac{\exp\left(\left(\boldsymbol{J}_l\delta\boldsymbol{\phi}\right)^{\wedge}\right)\exp\left(\boldsymbol{\phi}^{\wedge}\right)\boldsymbol{p} - \exp\left(\boldsymbol{\phi}^{\wedge}\right)\boldsymbol{p}}{\delta\boldsymbol{\phi}} \\
&\approx \lim_{\delta\boldsymbol{\phi}\to 0} \frac{\left(\boldsymbol{I}+\left(\boldsymbol{J}_l\delta\boldsymbol{\phi}\right)^{\wedge}\right)\exp\left(\boldsymbol{\phi}^{\wedge}\right)\boldsymbol{p} - \exp\left(\boldsymbol{\phi}^{\wedge}\right)\boldsymbol{p}}{\delta\boldsymbol{\phi}} \\
&= \lim_{\delta\boldsymbol{\phi}\to 0} \frac{\left(\boldsymbol{J}_l\delta\boldsymbol{\phi}\right)^{\wedge}\exp\left(\boldsymbol{\phi}^{\wedge}\right)\boldsymbol{p}}{\delta\boldsymbol{\phi}} \\
&= \lim_{\delta\boldsymbol{\phi}\to 0} \frac{-\left(\exp\left(\boldsymbol{\phi}^{\wedge}\right)\boldsymbol{p}\right)^{\wedge}\boldsymbol{J}_l\delta\boldsymbol{\phi}}{\delta\boldsymbol{\phi}} = -\left(\boldsymbol{R}\boldsymbol{p}\right)^{\wedge}\boldsymbol{J}_l.
\end{aligned}
$$

Lie Group and Lie Algebra

- Disturb model:

$$\begin{aligned}\frac{\partial (\boldsymbol{Rp})}{\partial \boldsymbol{\varphi}} &= \lim_{\boldsymbol{\varphi}\to 0} \frac{\exp\left(\boldsymbol{\varphi}^{\wedge}\right)\exp\left(\boldsymbol{\phi}^{\wedge}\right)\boldsymbol{p} - \exp\left(\boldsymbol{\phi}^{\wedge}\right)\boldsymbol{p}}{\boldsymbol{\varphi}} \\ &\approx \lim_{\boldsymbol{\varphi}\to 0} \frac{\left(1+\boldsymbol{\varphi}^{\wedge}\right)\exp\left(\boldsymbol{\phi}^{\wedge}\right)\boldsymbol{p} - \exp\left(\boldsymbol{\phi}^{\wedge}\right)\boldsymbol{p}}{\boldsymbol{\varphi}} \\ &= \lim_{\boldsymbol{\varphi}\to 0} \frac{\boldsymbol{\varphi}^{\wedge}\boldsymbol{Rp}}{\boldsymbol{\varphi}} = \lim_{\boldsymbol{\varphi}\to 0} \frac{-(\boldsymbol{Rp})^{\wedge}\boldsymbol{\varphi}}{\boldsymbol{\varphi}} = -(\boldsymbol{Rp})^{\wedge}.\end{aligned}$$

- More simple and clear
- In some literature we use operator $\oplus$ to denote this disturb model

$$\Delta R \oplus R = \exp(\delta\phi^{\wedge})R$$

Lie Group and Lie Algebra

- Disturb model in SE(3):

$$
\begin{aligned}
\frac{\partial (\boldsymbol{T}\boldsymbol{p})}{\partial \delta\boldsymbol{\xi}} &= \lim_{\delta\boldsymbol{\xi}\to 0} \frac{\exp\left(\delta\boldsymbol{\xi}^{\wedge}\right)\exp\left(\boldsymbol{\xi}^{\wedge}\right)\boldsymbol{p} - \exp\left(\boldsymbol{\xi}^{\wedge}\right)\boldsymbol{p}}{\delta\boldsymbol{\xi}} \\
&\approx \lim_{\delta\boldsymbol{\xi}\to 0} \frac{\left(\boldsymbol{I} + \delta\boldsymbol{\xi}^{\wedge}\right)\exp\left(\boldsymbol{\xi}^{\wedge}\right)\boldsymbol{p} - \exp\left(\boldsymbol{\xi}^{\wedge}\right)\boldsymbol{p}}{\delta\boldsymbol{\xi}} \\
&= \lim_{\delta\boldsymbol{\xi}\to 0} \frac{\delta\boldsymbol{\xi}^{\wedge}\exp\left(\boldsymbol{\xi}^{\wedge}\right)\boldsymbol{p}}{\delta\boldsymbol{\xi}} \\
&= \lim_{\delta\boldsymbol{\xi}\to 0} \frac{\begin{bmatrix} \delta\boldsymbol{\phi}^{\wedge} & \delta\boldsymbol{\rho} \\ \boldsymbol{0}^T & 0 \end{bmatrix}\begin{bmatrix} \boldsymbol{R}\boldsymbol{p}+\boldsymbol{t} \\ 1 \end{bmatrix}}{\delta\boldsymbol{\xi}} \\
&= \lim_{\delta\boldsymbol{\xi}\to 0} \frac{\begin{bmatrix} \delta\boldsymbol{\phi}^{\wedge}\left(\boldsymbol{R}\boldsymbol{p}+\boldsymbol{t}\right) + \delta\boldsymbol{\rho} \\ 0 \end{bmatrix}}{\delta\boldsymbol{\xi}} = \begin{bmatrix} \boldsymbol{I} & -\left(\boldsymbol{R}\boldsymbol{p}+\boldsymbol{t}\right)^{\wedge} \\ \boldsymbol{0}^T & \boldsymbol{0}^T \end{bmatrix} \triangleq (\boldsymbol{T}\boldsymbol{p})^{\odot}.
\end{aligned}
$$

Lie Group and Lie Algebra

- Conclusion
  - We can take the derivate via BCH formula
  - Disturb model is more simple and useful

Application: estimating camera pose

- Suppose we want to estimate the camera pose
- We have several observations from the projection function

$$s\begin{bmatrix} u \\ v \\ 1 \end{bmatrix}_i = K(RP_i + t) = KTP_i$$

- Minimizing the reprojection error: $(R,t)^* = T^* = \text{argmin}\frac{1}{2}\sum_{i=1}^{N} \|u_i - \pi(RP_i + t)\|_2^2$
  - Where $\pi(\cdot)$ is the projection equation (observation model)

Application: estimating camera pose

- Linearize the error: $e_i(x \oplus \Delta x) \approx e_i(x) + J(x)\Delta x$
- Derivative is defined by SE(3) disturb model:

$$\frac{\partial e}{\partial T} = \lim_{\delta\xi \to 0} \frac{e(\delta\xi \oplus T) - e(T)}{\delta\xi}$$

$$= \lim_{\delta\xi \to 0} \frac{\frac{1}{Z} K(\delta\xi \oplus T)P - \frac{1}{Z} KTP}{\delta\xi}$$

- Let $P' = TP$ then use chain rule: $\frac{\partial e}{\partial T} = \frac{\partial e}{\partial P'} \frac{\partial P'}{\partial T}$
- For P' we have:

$$\begin{bmatrix} su \\ sv \\ s \end{bmatrix} = \begin{bmatrix} f_x & 0 & c_x \\ 0 & f_y & c_y \\ 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} X' \\ Y' \\ Z' \end{bmatrix}.$$

$$u = f_x \frac{X'}{Z'} + c_x, \quad v = f_y \frac{Y'}{Z'} + c_y.$$

$$\frac{\partial \boldsymbol{e}}{\partial \boldsymbol{P'}} = -\begin{bmatrix} \frac{\partial u}{\partial X'} & \frac{\partial u}{\partial Y'} & \frac{\partial u}{\partial Z'} \\ \frac{\partial v}{\partial X'} & \frac{\partial v}{\partial Y'} & \frac{\partial v}{\partial Z'} \end{bmatrix} = -\begin{bmatrix} \frac{f_x}{Z'} & 0 & -\frac{f_x X'}{Z'^2} \\ 0 & \frac{f_y}{Z'} & -\frac{f_y Y'}{Z'^2} \end{bmatrix}.$$

Application: estimating camera pose

- The second item: $\frac{\partial(TP')}{\partial T} = \begin{bmatrix} I & -P'^{\wedge} \\ 0^T & 0^T \end{bmatrix}$
- Remove the homogeneous part: $\frac{\partial(TP')}{\partial T} = [I \quad -P'^{\wedge}]$

- Put them together:

$$\frac{\partial \boldsymbol{e}}{\partial \delta \boldsymbol{\xi}} = -\begin{bmatrix} \frac{f_x}{Z'} & 0 & -\frac{f_x X'}{Z'^2} & -\frac{f_x X' Y'}{Z'^2} & f_x + \frac{f_x X^2}{Z'^2} & -\frac{f_x Y'}{Z'} \\ 0 & \frac{f_y}{Z'} & -\frac{f_y Y'}{Z'^2} & -f_y - \frac{f_y Y'^2}{Z'^2} & \frac{f_y X' Y'}{Z'^2} & \frac{f_y X'}{Z'} \end{bmatrix}.$$

Application: estimating camera pose

- If we want to take the derivative of Point P

$$s\begin{bmatrix} u \\ v \\ 1 \end{bmatrix}_i = K(RP_i + t) = KTP_i$$

$$\frac{\partial e}{\partial P} = \frac{\partial e}{\partial P'}\frac{\partial P'}{\partial P} = -\begin{bmatrix} f_x/Z' & 0 & -f_x X'/Z'^2 \\ 0 & f_y/Z' & -f_y Y'/Z'^2 \end{bmatrix} R$$

- P is not relevant to translation t

Application: estimating camera pose

- We can also compute these Jacobians in SO(3)
- With Jacobian in manifold it will be easy to perform Gauss-Newton iterations to solve the camera's motion iteratively

Any Questions?